DiagramTec

Informe del proyecto

Planificación de Sitio Web

Profesor: Diego Callamullo

Materia: PISWD

Orientación: Programación

Integrantes del equipo de desarrollo:

- 1°Lider Back-End: Maite Vázquez
- 2°Lider y Marketing: Nayla Gómez
- Front-End: Santiago Álvarez – Leandro Pereyra
- Documentación/Base de Datos: Joaquín Villalva

Profesor: Callamullo Diego
Orientación: Programación.
Institución: E.E.S.T. N°1 Monte Grande

Fecha de entrega FINAL: 04/07/2026

Versión: 3.0

Indice

Introducción 5
Nombre y Concepto 6
Justificación de los elementos del logotipo 6
Justificación de la paleta de colores 7
Objetivo del Sitio 8
Público Objetivo 8
Problema que Resuelve / Servicio que Ofrece 9
Idea del Proyecto 10
Temática del Sitio 11
Bocetos - Marquetado 12
Estructura Base de Datos 20
Diseño Base de Datos 21
Validaciones del Sistema 22
Sector Frontend 23
Arquitectura utilizada 29
Estructura general del Frontend 29
Función de las páginas 30
Función de los componentes 32
Ventajas de la arquitectura basada en componentes 33
Flujo general de la arquitectura Frontend 35
Tecnologías utilizadas 43
Información general del sector 45
Funciones principales del backend 45
Funcionamiento general 46
Arquitectura utilizada 47
Sector de Documentación 63
Funcionamiento de la base de datos 72
Cómo interactúan los sectores 84
Responsabilidades principales del Backend 85
Base de datos 86
Responsabilidades principales de la base de datos 86

Función de MySQL dentro del sistema 87
Comunicación completa entre sectores 87
Beneficios de la separación por sectores..... 88
Restricciones SQL 96
SQL utilizado para la creación de la Base de Datos 98
Diagrama DER (Entidad Relación)..... 102
Relaciones: 102
Diccionario de Datos 103
Derechos de Autor y Propiedad Intelectual 107

Introducción

En la actualidad, la representación de procesos mediante herramientas visuales se ha vuelto fundamental en el ámbito educativo, especialmente en áreas relacionadas con la informática y el análisis de sistemas. Los Diagramas de Flujo de Datos (DFD) permiten modelar de manera clara cómo circula la información dentro de un sistema, facilitando su comprensión, análisis y desarrollo. Sin embargo, en muchos entornos educativos, estos diagramas suelen manejarse de forma estática, desorganizada y sin posibilidades de interacción o edición dinámica.

El presente proyecto propone el desarrollo de una plataforma web orientada a la gestión, creación y edición de Diagramas de Flujo de Datos dentro de una institución educativa. El sistema permitirá a los profesores crear clases virtuales y compartir diagramas mediante un código de acceso, brindando a los alumnos la posibilidad de visualizar, descargar y generar sus propias versiones editables a partir de los materiales proporcionados.

Una de las características más destacadas del sistema es la integración de un módulo de procesamiento que, a partir de una imagen de un diagrama, permite reconocer sus elementos y transformarlos en una estructura editable. Esto facilita la reutilización de diagramas existentes y promueve un aprendizaje más dinámico e interactivo, al permitir a los estudiantes trabajar directamente sobre representaciones previamente creadas.

Asimismo, el sistema contempla distintos tipos de usuarios, cada uno con funcionalidades específicas, lo que garantiza una organización adecuada de la información y un uso controlado de la plataforma. Los profesores podrán gestionar sus clases y contenidos, mientras que los alumnos podrán interactuar con los diagramas sin alterar los originales, generando copias propias para su edición y almacenamiento.

En este contexto, la implementación de esta plataforma busca optimizar la enseñanza y el aprendizaje de conceptos relacionados con el flujo de datos, proporcionando una herramienta moderna, accesible y eficiente que centraliza la información y mejora la experiencia educativa dentro de la institución.

Nombre y Concepto

El nombre “Diagram Tec” es adecuado porque combina dos elementos clave del proyecto:

“Diagram” hace referencia directa a la función principal del sistema: la creación de diagramas de flujo, lo que facilita que cualquier usuario entienda inmediatamente el propósito de la plataforma.

“Tec” representa la identidad institucional de la escuela, reforzando el sentido de pertenencia y dejando claro que es una herramienta diseñada específicamente para la comunidad del Tecnológico.

En conjunto, el nombre es claro, directo y fácil de recordar, además de transmitir tanto funcionalidad como identidad.

Justificación de los elementos del logotipo

El logotipo presenta un diseño minimalista con dos componentes principales:

Tipografía “Geomath3”:

Se eligió esta fuente por su estilo limpio, geométrico y moderno. Esto transmite:

- Orden y estructura (relacionado con los diagramas de flujo)
- Claridad visual
- Enfoque tecnológico
- Elemento gráfico (forma redondeada tipo botón/etiqueta):
- La figura que encierra “Tec” simula:
- Un nodo o bloque dentro de un diagrama de flujo
- Un botón interactivo, haciendo alusión a una herramienta digital

Además, las esquinas redondeadas suavizan el diseño, haciéndolo más amigable y accesible para el usuario.

Flecha:

Representa el flujo, dirección y secuencia lógica, elementos esenciales en los diagramas de flujo.



En conjunto, estos elementos comunican tecnología, funcionalidad y dinamismo.

Justificación de la paleta de colores

La paleta utilizada (#FEF9E1, #E5D0AC, #A31D1D, #6D2323) tiene un significado tanto visual como institucional:



Colores claros (#FEF9E1 y #E5D0AC):

Aportan limpieza, claridad y simplicidad

Facilitan la lectura y reducen la fatiga visual

Funcionan como fondo neutro que resalta los elementos importantes

Rojo y bordó (#A31D1D y #6D2323):

Representan la identidad del Tecnológico

Transmiten seriedad, formalidad y pertenencia institucional

Generan contraste visual, destacando elementos clave como el “Tec”

En conjunto, la paleta logra un equilibrio entre profesionalismo, identidad escolar y legibilidad.

Objetivo del Sitio

El objetivo principal del sitio web es desarrollar una plataforma digital que permita la creación, gestión y edición de Diagramas de Flujo de Datos (DFD) de manera interactiva, facilitando su uso dentro del entorno educativo. Se busca proporcionar una herramienta accesible que centralice los diagramas utilizados en la institución, permitiendo a los profesores compartir contenido y a los alumnos trabajar sobre él de forma organizada.

Además, el sistema tiene como finalidad mejorar el proceso de enseñanza y aprendizaje mediante la incorporación de funcionalidades que permitan transformar diagramas en formato imagen en estructuras editables, promoviendo un enfoque más dinámico y práctico. De esta manera, se pretende optimizar la comprensión de los procesos de información y fomentar el desarrollo de habilidades en el análisis de sistemas.

En conjunto, el sitio busca integrar tecnología y educación, ofreciendo una solución moderna que facilite la interacción con los diagramas, mejore la organización de los contenidos y contribuya a una experiencia de aprendizaje más eficiente.

Público Objetivo

El sitio web está dirigido principalmente a la comunidad educativa del Tecnológico, especialmente a estudiantes y docentes del área de informática y análisis de sistemas. Los estudiantes constituyen el público principal, ya que utilizarán la plataforma para acceder a diagramas, trabajar sobre ellos y desarrollar sus propias versiones como parte de su aprendizaje.

Por otro lado, los docentes también forman parte fundamental del público objetivo, dado que serán los encargados de crear clases, compartir materiales y guiar el uso de la plataforma dentro del proceso educativo. La herramienta les permitirá organizar mejor los contenidos y facilitar la interacción con los alumnos.

De manera secundaria, el sistema puede ser utilizado por cualquier persona interesada en la creación y comprensión de Diagramas de Flujo de Datos, aunque su enfoque principal está orientado al uso académico dentro de la institución.

Problema que Resuelve / Servicio que Ofrece

En el ámbito educativo, los Diagramas de Flujo de Datos suelen manejarse de manera poco eficiente, ya que generalmente se encuentran en formatos estáticos como imágenes o documentos que dificultan su edición, reutilización y organización. Esto genera problemas al momento de trabajar con ellos, especialmente cuando los estudiantes necesitan modificarlos o comprender su estructura en profundidad.

Además, el uso de múltiples herramientas externas para crear y editar diagramas provoca una falta de integración dentro del proceso de enseñanza, dificultando el acceso centralizado a los materiales y la interacción entre docentes y alumnos. Esto puede afectar la claridad en la transmisión de los contenidos y limitar las posibilidades de aprendizaje práctico.

El proyecto propone solucionar estas problemáticas mediante una plataforma web que centraliza la gestión de los diagramas, permitiendo su almacenamiento, visualización y edición en un mismo entorno. A través del uso de clases con códigos de acceso, se facilita la organización del contenido y la participación de los alumnos.

Asimismo, el sistema incorpora la posibilidad de transformar imágenes de diagramas en estructuras editables, lo que permite reutilizar materiales existentes y trabajar sobre ellos de manera dinámica. También se garantiza que los alumnos puedan generar copias de los diagramas originales sin modificarlos, preservando así la integridad del contenido compartido por los docentes.

De esta manera, el sitio ofrece un servicio que mejora la accesibilidad, organización y comprensión de los Diagramas de Flujo de Datos, brindando una herramienta práctica y eficiente para el entorno educativo.

Idea del Proyecto

El proyecto consiste en el desarrollo de una plataforma web educativa orientada a la creación, gestión y edición de Diagramas de Flujo de Datos (DFD) dentro de una institución académica. Este sitio busca modernizar la forma en que se trabajan los diagramas en el ámbito educativo, pasando de un enfoque tradicional y estático a uno dinámico e interactivo, facilitando tanto la enseñanza como el aprendizaje de conceptos relacionados con el análisis de sistemas.

Se trata de una plataforma educativa y herramienta web interactiva, similar a un aula virtual, diseñada específicamente para trabajar con diagramas de flujo de datos. Su objetivo principal es facilitar la creación, visualización y edición de estos diagramas, permitiendo a los usuarios operar de manera organizada, accesible y eficiente. Además, busca centralizar todos los diagramas utilizados dentro de la institución, mejorando la gestión de los contenidos y optimizando la experiencia educativa.

El sitio está dirigido principalmente a estudiantes de informática o áreas afines, docentes que trabajen con análisis de sistemas y, en general, a la comunidad educativa del Tecnológico. Asimismo, puede ser utilizado por cualquier persona interesada en comprender y desarrollar diagramas de flujo de datos.

Actualmente, en muchos entornos educativos, los diagramas se manejan de forma desorganizada, en formatos no editables o mediante herramientas externas que no están integradas al proceso de aprendizaje. Esto dificulta su modificación, reutilización y comprensión. El proyecto propone solucionar estas problemáticas mediante una plataforma que permita almacenar los diagramas de manera centralizada, organizar el acceso a través de clases con códigos, y ofrecer herramientas de edición digital.

Una de las funcionalidades más relevantes del sistema es la posibilidad de generar copias de los diagramas originales subidos por los profesores, permitiendo que los alumnos trabajen sobre sus propias versiones sin alterar el contenido base. Además, se incorpora un módulo capaz de procesar imágenes de diagramas y transformarlas en estructuras editables, lo que amplía las posibilidades de uso y facilita el aprovechamiento de materiales existentes.

De esta manera, la plataforma ofrece un servicio completo que mejora la accesibilidad, organización y comprensión de los diagramas de flujo de datos, brindando un entorno moderno, práctico e interactivo para el desarrollo de actividades académicas.

Temática

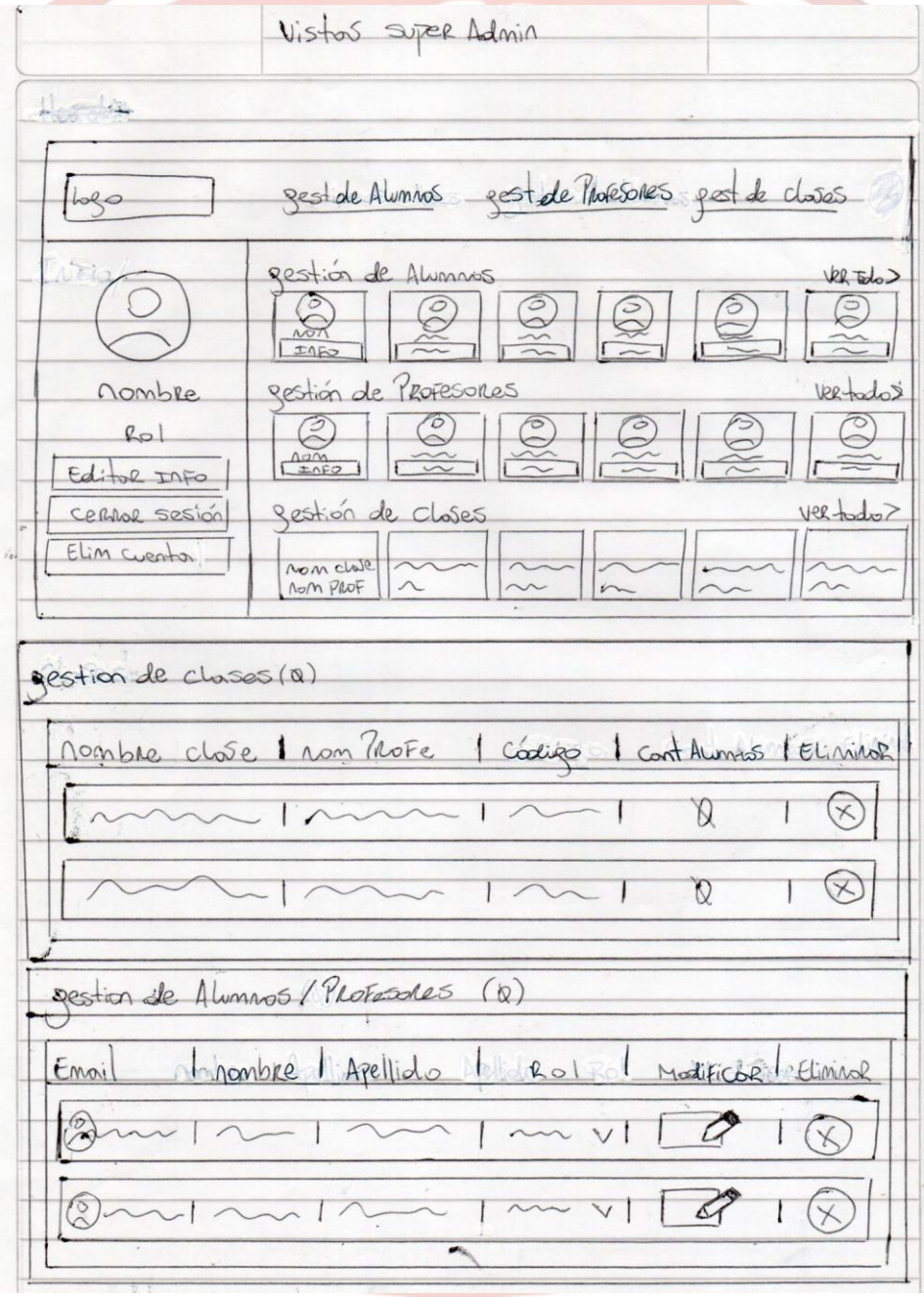
La temática del sitio se centra en el ámbito educativo y tecnológico, específicamente en el área de análisis de sistemas y representación de procesos mediante Diagramas de Flujo de Datos (DFD). La plataforma está orientada a brindar un entorno digital que facilite el aprendizaje, la práctica y la comprensión de estos diagramas, integrando herramientas interactivas que permiten su creación y edición.

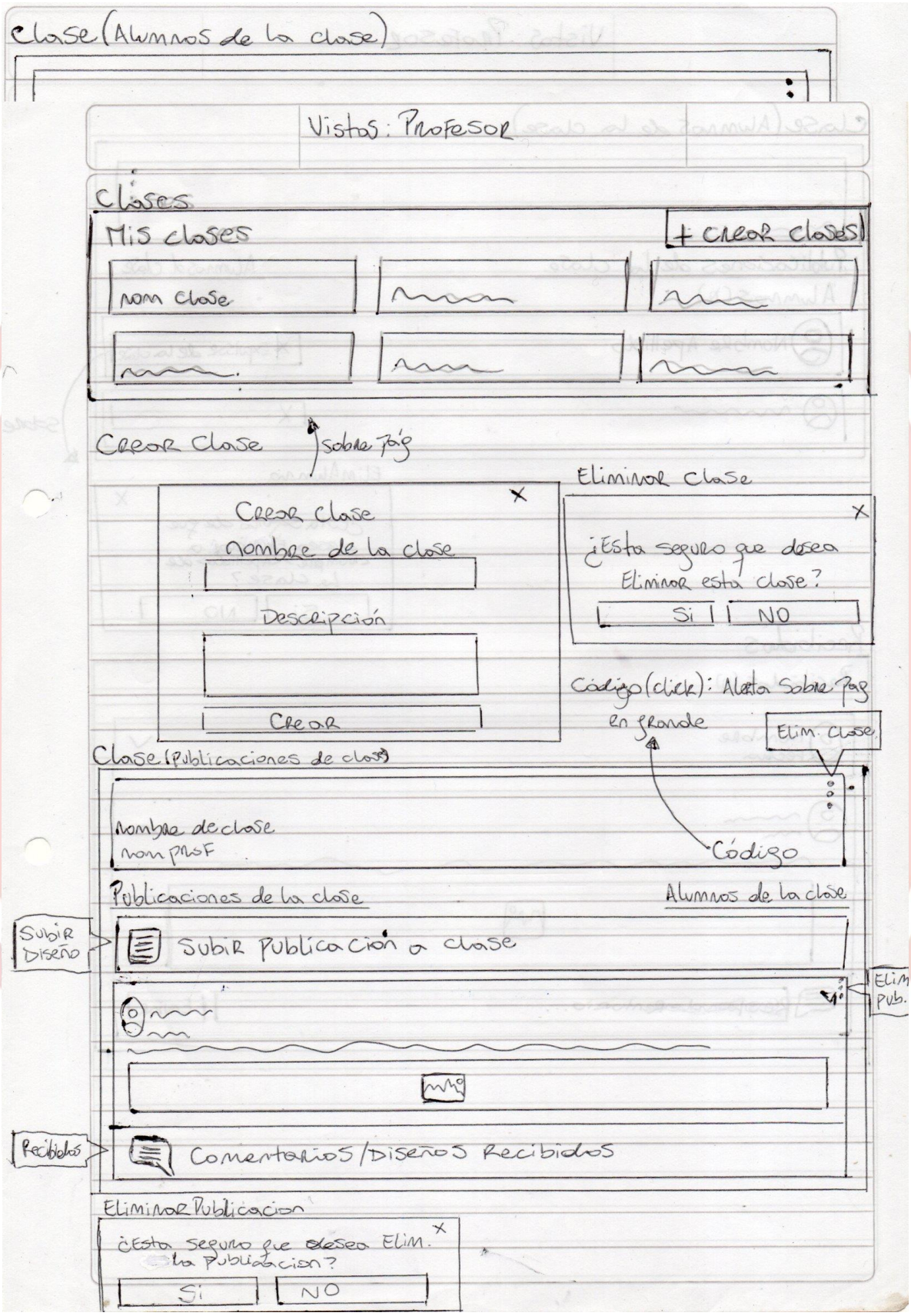
El enfoque del sitio combina elementos académicos con recursos tecnológicos, promoviendo una experiencia dinámica y accesible para los usuarios. Se busca que los estudiantes no solo visualicen los diagramas, sino que también puedan interactuar con ellos, modificarlos y generar sus propias versiones, favoreciendo así un aprendizaje activo.

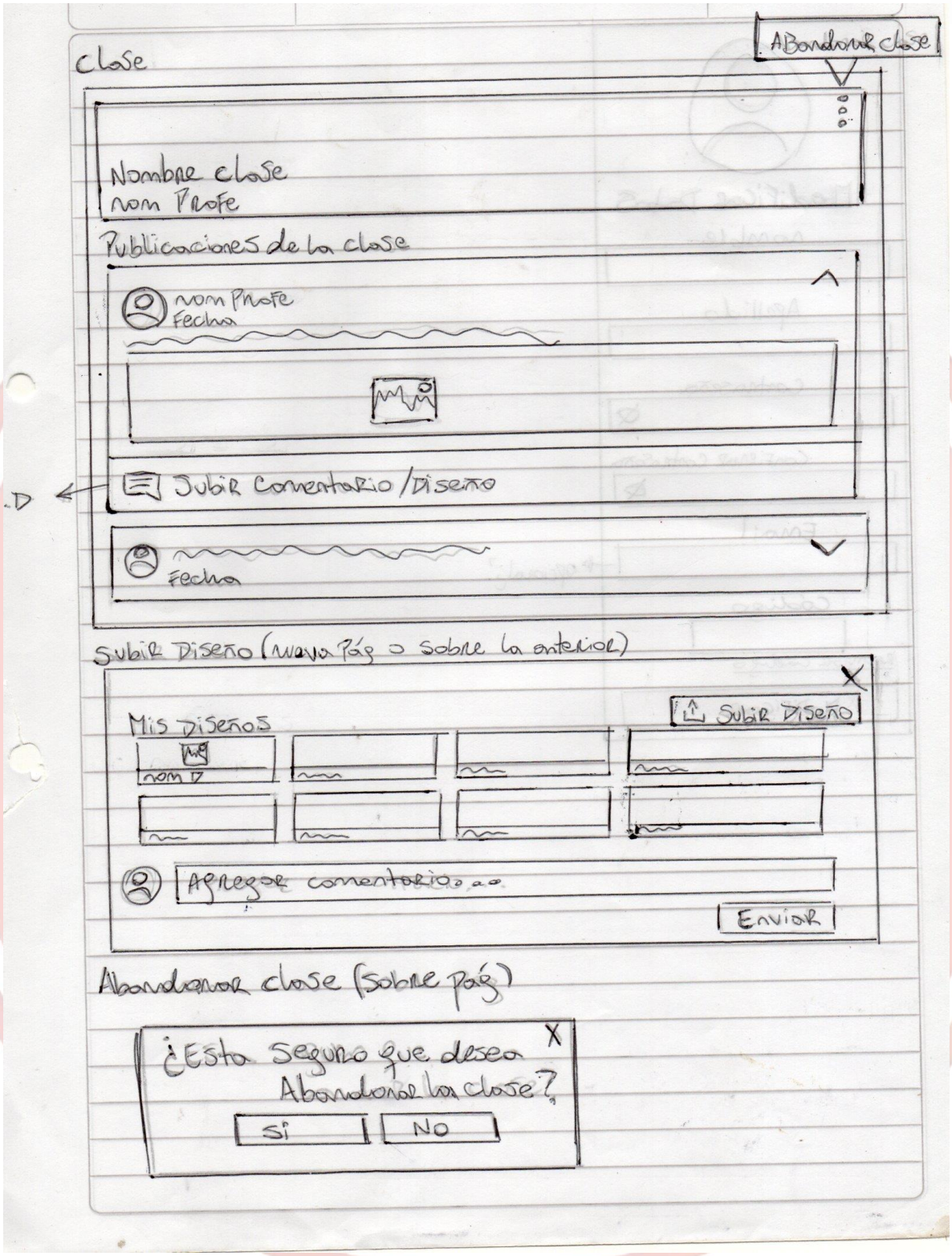
Asimismo, la temática responde a una identidad institucional, ya que el sistema está diseñado para ser utilizado dentro del contexto del Tecnológico, manteniendo una coherencia con su perfil educativo y sus necesidades específicas. En este sentido, la plataforma no solo cumple una función técnica, sino también pedagógica, al servir como apoyo en el desarrollo de competencias relacionadas con el modelado de sistemas y el manejo de información.

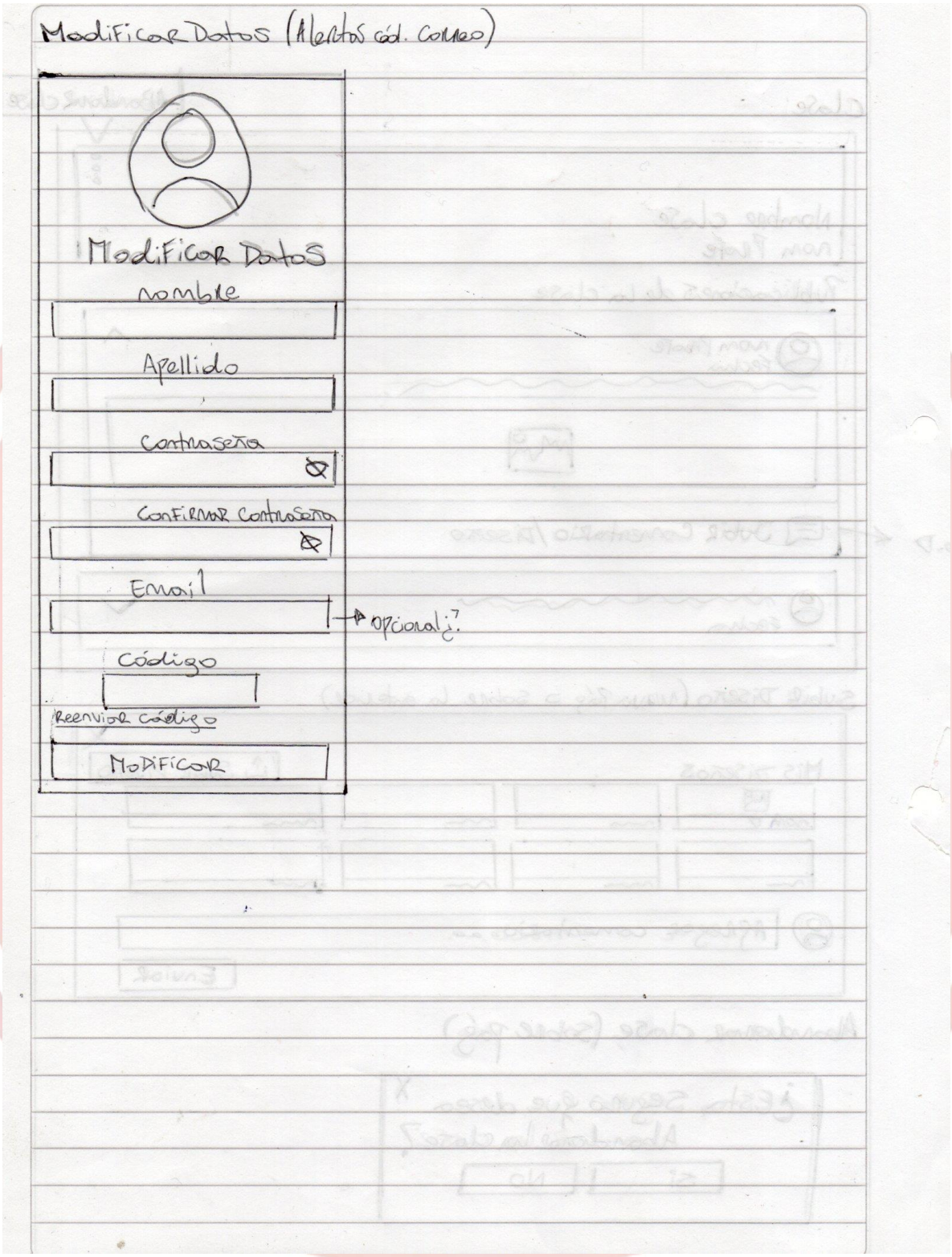
Bocetos - Marquetado

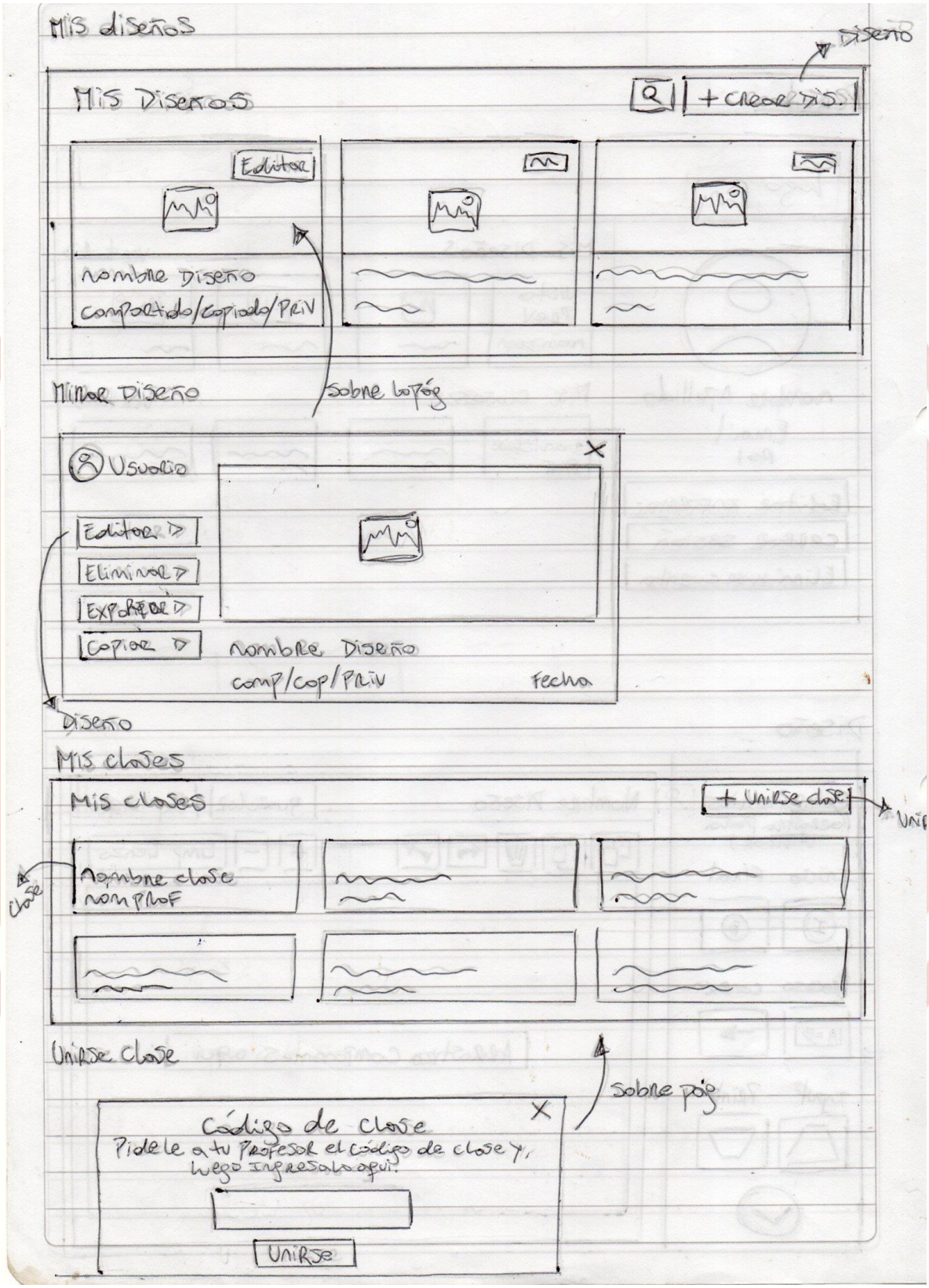
Se realizaron bocetos iniciales del diseño del sitio web con el objetivo de definir la estructura visual, la distribución de los elementos y la navegación entre las distintas secciones. Estos bocetos representan una primera versión del diseño (v1), permitiendo visualizar de forma general cómo será la interfaz del sistema.

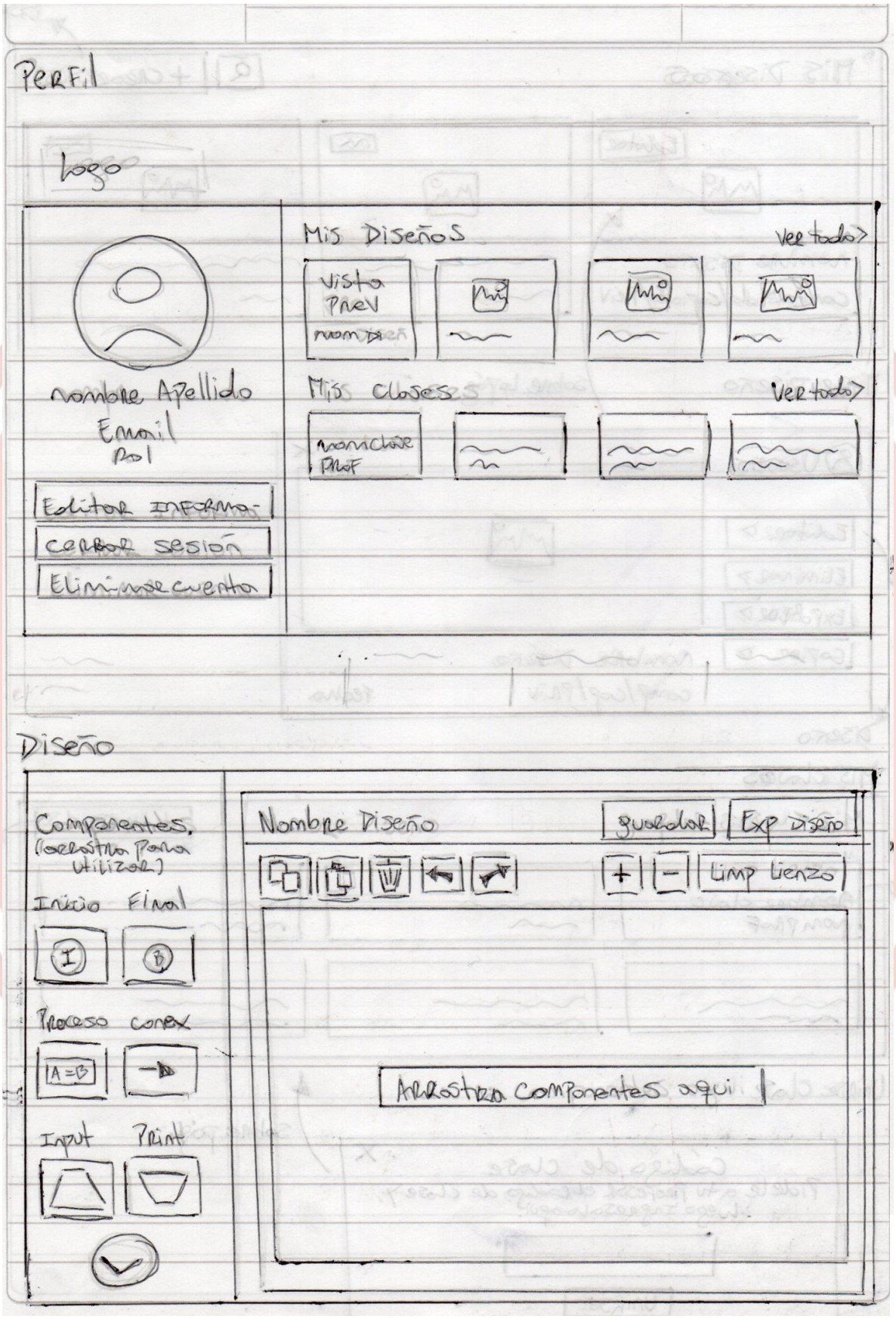


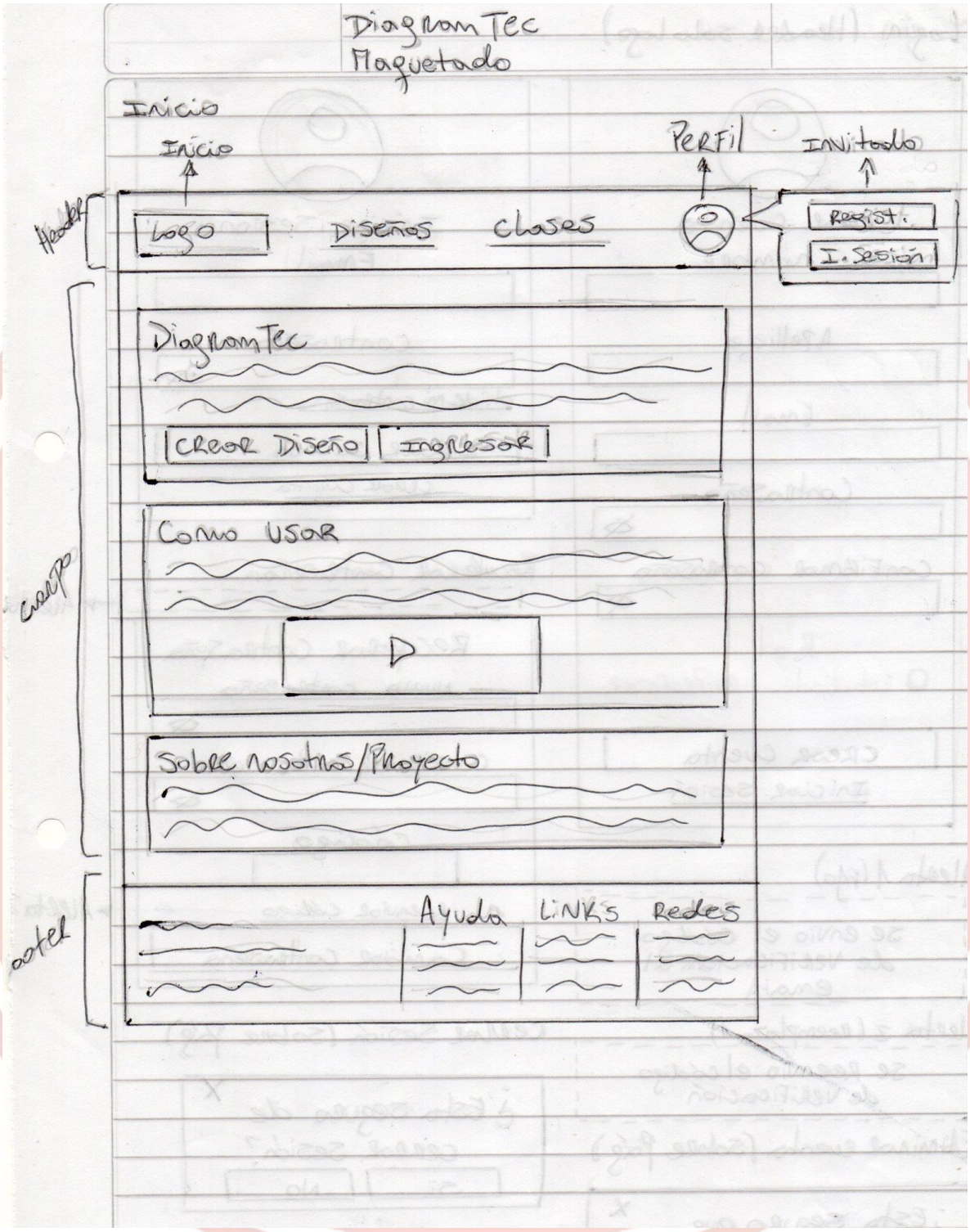

Clase (Alumnos de la clase)
Vistas: Profesor
Clases
Mis clases
+ Crear clase!
nom clase
Crear Clase
sobre pág
Crear Clase
nombre de la clase
Descripción
Crear
Eliminar Clase
¿Esta seguro que desea Eliminar esta clase?
Si
NO
Código (click): Alerta Sobre Pág en grande
Elim. Clase
Clase (Publicaciones de clase)
nombre de clase
nom PROF
Código
Publicaciones de la clase
Alumnos de la clase
Subir Diseño
Subir Publicación a clase
Elim Pub.
img
Recibidos
Comentarios/Diseños Recibidos
Eliminar Publicacion
¿Esta seguro que desea Elim. la Publicacion?
Si
NO


clase
Abandonar clase
Nombre clase
nom Profe
Publicaciones de la clase
nom Profe
Fecha
Subir Comentario / Diseño
Fecha
Subir Diseño (nueva Pág o sobre la anterior)
Mis Diseños
Subir Diseño
nom D
Agregar comentario...
Enviar
Abandonar clase (Sobre Pág)
¿Esta seguro que desea Abandonar la clase?
SI
NO

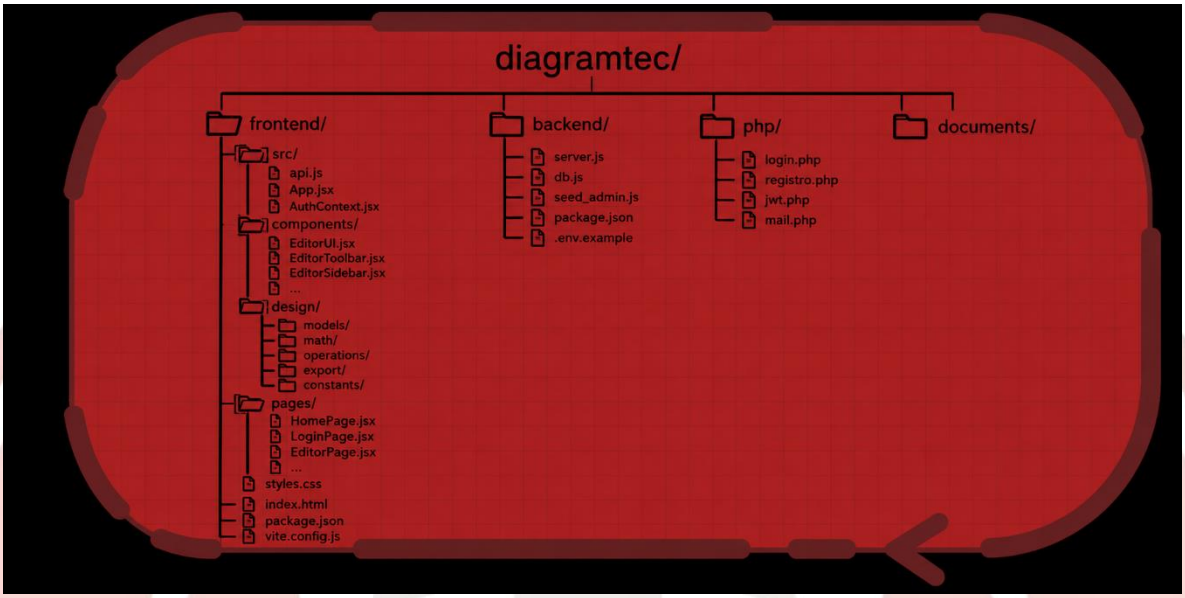

Modificar Datos (Alertas cód. Correo)
Modificar Datos
nombre
Apellido
Contraseña
Confirmar Contraseña
Email
opcional¿?
Código
Reenviar código
Modificar


Mis diseños
Diseño
Mis Diseños
+ Crear Dis.
Editar
nombre Diseño
compartido/copiado/PRIV
Mirar Diseño
sobre bpág
Usuario
Editar
Eliminar
Exportar
Copiar
nombre Diseño
comp/cop/PRIV
Fecha
Diseño
Mis clases
Mis clases
+ Unirse clase
nombre clase
nom PROF
Unirse Clase
sobre pág
Código de clase
Pídele a tu Profesor el código de clase y, luego ingresalo aquí.
UNIRSE


Perfil
Logo
Mis Diseños
Ver todo>
Vista Prev
Mis clases
Ver todo>
nombre Apellido
Email
Rol
Editar Informa-
cerrar sesión
Eliminar cuenta
Diseño
Componentes.
(arrastra para utilizar)
Inicio
Final
Proceso
conex
Input
Print
Nombre Diseño
guardar
Exp diseño
Limp lienzo
Arrastra Componentes aqui


DiagramTec
Maquetado
Inicio
Inicio
Perfil
Invitado
Header
Logo
Diseños
Clases
Regist.
I. Sesión
DiagramTec
Crear Diseño
Ingresar
Como USAR
Sobre nosotros/Proyecto
Ayuda
LINKS
Redes

Estructura del sitio



Estructura Base de Datos

La base de datos del sistema ha sido diseñada con el objetivo de organizar y gestionar de manera eficiente la información relacionada con los usuarios, las clases y los diagramas de flujo de datos. Su estructura se basa en un modelo relacional que permite mantener la integridad de los datos y establecer relaciones claras entre las distintas entidades del sistema.

En primer lugar, la tabla usuarios almacena la información de todas las personas que interactúan con la plataforma. En ella se registran datos como el nombre, correo electrónico, contraseña y tipo de usuario, permitiendo diferenciar entre profesores y alumnos. Además, se incluye un campo de fecha de creación que permite llevar un control temporal de los registros.

Por otro lado, la tabla clases representa los espacios virtuales creados por los profesores. Cada clase cuenta con un nombre, un código de acceso único y una referencia al profesor que la creó. Este código permite que los alumnos puedan inscribirse de manera sencilla, replicando el funcionamiento de plataformas educativas modernas.

La relación entre alumnos y clases se gestiona mediante la tabla inscripciones, la cual actúa como una tabla intermedia. Esta estructura permite que un alumno pueda estar inscrito en múltiples clases y que cada clase tenga varios alumnos, estableciendo así una relación de muchos a muchos.

En cuanto a los diagramas, la tabla diagramas es una de las más importantes del sistema. En ella se almacenan tanto los diagramas originales subidos por los profesores como las copias generadas por los alumnos. Cada registro incluye información como el nombre del diagrama, el usuario que lo creó, la clase a la que pertenece, la ruta de la imagen original y los datos editables en formato JSON. Además, se incorpora un campo que referencia al diagrama original en caso de tratarse de una copia, lo que permite mantener una relación entre versiones sin modificar el archivo base.

Finalmente, todas las tablas principales incluyen un campo de fecha de creación, lo que facilita el seguimiento de la información, el ordenamiento de los datos y la gestión temporal dentro del sistema.

En conjunto, esta estructura permite un manejo organizado, escalable y eficiente de la información, adaptándose a las necesidades del proyecto y garantizando una correcta interacción entre los distintos componentes de la plataforma.

Diseño Base de Datos

El diseño de la base de datos fue realizado siguiendo un modelo relacional, con el objetivo de garantizar una correcta organización de la información, evitar redundancias y facilitar la escalabilidad del sistema. La estructura se basa en entidades principales como usuarios, clases y diagramas, junto con una tabla intermedia que permite gestionar relaciones complejas de manera eficiente.

En primer lugar, la tabla users centraliza la información de todos los usuarios del sistema, diferenciando sus roles mediante el atributo role. Esta decisión permite manejar distintos tipos de usuarios (docentes y alumnos) dentro de una misma estructura, simplificando el diseño y evitando la necesidad de múltiples tablas separadas.

La tabla classes representa los espacios virtuales creados por los profesores. Se incorpora un campo code único que permite el acceso a las clases mediante un sistema similar a plataformas educativas actuales, facilitando la inscripción de alumnos sin necesidad de procesos complejos.

Para resolver la relación entre usuarios y clases, se implementa la tabla class_members, la cual actúa como entidad intermedia. Esta decisión responde a la necesidad de representar una relación de muchos a muchos, donde un usuario puede pertenecer a varias clases y una clase puede tener múltiples usuarios. Además, esta tabla permite registrar información adicional como la fecha de ingreso (joinedAt).

En cuanto a la gestión de diagramas, la tabla designs fue diseñada para almacenar tanto los diagramas originales como sus copias. Se incluye el campo content en formato JSON, lo que permite guardar estructuras flexibles y editables del diagrama. Esto resulta fundamental para la funcionalidad del editor dentro del sistema.

Asimismo, se incorporan los campos isCopy y originalId con el objetivo de gestionar la relación entre diagramas originales y sus copias. Esta decisión permite que los alumnos trabajen sobre versiones propias sin modificar el contenido base creado por los profesores, manteniendo la integridad de la información.

Por último, todas las tablas incluyen un campo createdAt, lo que permite llevar un control temporal de los registros, facilitando la organización, el seguimiento y posibles futuras funcionalidades como historiales o filtros por fecha.

En conjunto, este diseño permite una estructura clara, eficiente y adaptable, alineada con los requerimientos del sistema y preparada para futuras ampliaciones.

Validaciones del Sistema

Con el objetivo de garantizar la integridad, consistencia y seguridad de los datos almacenados en la base de datos, se establecieron diversas validaciones tanto a nivel de base de datos como a nivel lógico del sistema.

En primer lugar, se aplican restricciones de unicidad en campos clave. El atributo *email* en la tabla *users* es único, evitando que existan múltiples cuentas con el mismo correo electrónico. De igual manera, el campo *code* en la tabla *classes* también es único, asegurando que cada clase pueda identificarse mediante un código exclusivo para el acceso de los alumnos.

Además, se implementan claves foráneas (*foreign keys*) para garantizar la integridad referencial entre las tablas. Estas relaciones aseguran que no se puedan crear registros que hagan referencia a entidades inexistentes. Por ejemplo, un registro en *class_members* no puede existir si el usuario o la clase asociada no están previamente registrados en el sistema.

También se incorporan restricciones de no nulidad (*NOT NULL*) en campos esenciales como nombres, correos electrónicos, contraseñas y títulos, evitando registros incompletos que puedan afectar el funcionamiento del sistema.

En la tabla *class_members*, se define una restricción única compuesta (*UNIQUE*) sobre los campos *classId* y *userId*, lo que impide que un mismo usuario se registre más de una vez en la misma clase, evitando duplicaciones.

En cuanto a la tabla *designs*, se establecen validaciones para el manejo de copias de diagramas. El campo *isCopy* indica si un diagrama es una copia, mientras que *originalId* referencia al diagrama base. Esto permite controlar la relación entre versiones y asegurar que las copias mantengan un vínculo válido con su origen.

A nivel lógico del sistema, se pueden aplicar validaciones adicionales como la verificación del formato del correo electrónico, la longitud mínima de contraseñas, y la validación del código de acceso a clases antes de permitir la inscripción de un usuario.

En conjunto, estas validaciones permiten mantener la coherencia de los datos, prevenir errores y asegurar un funcionamiento confiable del sistema.

Sector Frontend

Información general del sector

El sector Frontend de DiagramTec fue desarrollado con el objetivo de brindar una interfaz moderna, intuitiva y dinámica para que los usuarios puedan interactuar fácilmente con la plataforma. Este sector es el encargado de mostrar toda la información visual del sistema y permitir la interacción entre el usuario y las funcionalidades principales.

La aplicación frontend permite:

- Registro e inicio de sesión de usuarios.
- Gestión de clases.
- Creación y edición de diagramas.
- Exportación de diagramas.
- Visualización de comentarios.
- Administración de usuarios y contenido.

Todo el sistema visual fue diseñado para funcionar como una SPA (Single Page Application), evitando recargas constantes y mejorando la experiencia del usuario.

Desarrollo realizado en el sector Frontend

El frontend fue desarrollado utilizando React junto con Vite como entorno de desarrollo, permitiendo una aplicación rápida, modular y optimizada. Durante el desarrollo se priorizó la experiencia del usuario, la velocidad de navegación y la organización visual de cada sección del sistema.

La estructura visual fue dividida en múltiples páginas y componentes reutilizables, lo que permitió mantener un código más limpio, escalable y fácil de mantener.

Sistema de navegación

Se implementó un sistema de navegación dinámica mediante React Router DOM, permitiendo que el usuario pueda desplazarse entre las distintas secciones sin necesidad de recargar la página.

Las rutas fueron organizadas para separar las funcionalidades principales del sistema:

- Página principal.
- Inicio de sesión.
- Registro de usuarios.
- Editor de diagramas.
- Gestión de clases.
- Perfil de usuario.
- Panel administrador.
- Página de error 404.

Además, algunas rutas fueron protegidas mediante autenticación para impedir el acceso a usuarios no autorizados.

Interfaz visual y experiencia de usuario

La interfaz fue diseñada buscando simplicidad, claridad visual y facilidad de uso. Para ello se utilizaron estilos personalizados mediante CSS, organizando correctamente colores, botones, menús y paneles.

Se trabajó especialmente en:

- Diseño responsive.
- Distribución organizada de herramientas.
- Botones intuitivos.
- Formularios claros.
- Navegación sencilla.
- Adaptación visual entre secciones.

El objetivo fue que cualquier usuario pudiera utilizar el sistema sin necesidad de conocimientos técnicos avanzados.

Sistema de autenticación visual

El frontend implementa formularios de autenticación para:

- Registro de usuarios.
- Inicio de sesión.
- Cierre de sesión.

Estos formularios se conectan directamente con el backend mediante Axios, enviando solicitudes HTTP hacia la API.

También se desarrolló un sistema de persistencia de sesión utilizando Context API, permitiendo:

- Mantener al usuario autenticado.
- Compartir información entre componentes.
- Controlar permisos.
- Proteger rutas privadas.

Editor de diagramas

Uno de los módulos más importantes desarrollados en el frontend fue el editor visual de diagramas.

El editor permite:

- Crear diagramas dinámicamente.
- Agregar figuras.
- Mover elementos.
- Editar contenido.
- Organizar componentes visuales.
- Exportar diagramas.

Para esto se desarrollaron componentes específicos encargados del renderizado y manipulación visual de los elementos gráficos.

El editor funciona completamente del lado del cliente, ofreciendo una experiencia fluida e interactiva.

Exportación de contenido

El sistema permite exportar diagramas en distintos formatos.

Para ello se utilizaron librerías como:

- html2canvas
- jsPDF
- dom-to-image-more

Estas tecnologías permiten:

- Capturar el diagrama visual.
- Convertir contenido HTML a imagen.
- Generar archivos PDF.
- Descargar archivos automáticamente.

Esto facilita que los usuarios puedan guardar o compartir sus trabajos.

Gestión de clases y usuarios

Desde el frontend también se desarrollaron funcionalidades relacionadas con la administración académica del sistema.

Los usuarios pueden:

- Crear clases.
- Unirse mediante códigos.
- Ver miembros.
- Administrar diagramas.
- Publicar comentarios.

Además, el panel administrador permite gestionar contenido y usuarios desde una interfaz centralizada.

Comunicación con el Backend

El frontend se conecta constantemente con el backend mediante Axios.

Las solicitudes permiten:

- Obtener datos.
- Crear registros.
- Editar información.
- Eliminar contenido.
- Validar usuarios.

Toda la comunicación se realiza utilizando APIs REST y respuestas en formato JSON.

Esto permite mantener una separación clara entre la capa visual y la lógica del sistema.

Organización del código Frontend

El proyecto fue organizado siguiendo una estructura modular.

Las carpetas principales fueron separadas en:

Carpeta	Función
pages	Contiene las vistas principales
components	Componentes reutilizables
context	Manejo de autenticación y estados globales
services/api	Conexión con backend
assets	Recursos visuales
styles	Archivos CSS

Esta organización mejora:

- Escalabilidad.
- Mantenimiento.
- Reutilización de código.
- Claridad del proyecto.

Objetivo del Frontend

El principal objetivo del sector frontend fue crear una plataforma visualmente organizada, rápida y fácil de utilizar, permitiendo que los usuarios puedan interactuar con diagramas y clases de manera eficiente.

Se buscó combinar:

- Buen rendimiento.
- Interfaz moderna.
- Navegación fluida.
- Organización visual.
- Facilidad de uso.
- Integración completa con el backend.

El frontend de DiagramTec fue desarrollado utilizando una arquitectura modular basada en componentes mediante React. Esta arquitectura permite dividir el sistema en múltiples partes independientes y reutilizables, facilitando el mantenimiento, la escalabilidad y la organización general del proyecto.

La estructura fue diseñada siguiendo una separación clara de responsabilidades, donde cada archivo cumple una función específica dentro de la aplicación.

Estructura general del Frontend

El proyecto frontend se encuentra organizado en distintas carpetas y módulos que trabajan conjuntamente para construir la interfaz visual y gestionar la interacción con el usuario.

La arquitectura principal se divide en:

- Pages
- Components
- Context
- Services/API
- Assets
- Styles

Cada uno de estos sectores cumple una función fundamental dentro del funcionamiento de la aplicación.

Pages

Las páginas representan las vistas principales del sistema y funcionan como puntos de entrada para cada funcionalidad de la plataforma.

Cada página contiene la lógica principal de una sección específica y organiza los componentes visuales necesarios para su funcionamiento.

Archivos principales

Archivo	Función
HomePage.jsx	Página principal del sistema
LoginPage.jsx	Inicio de sesión de usuarios
SignupPage.jsx	Registro de nuevos usuarios
EditorPage.jsx	Editor principal de diagramas
ClassesPage.jsx	Visualización de clases
ClassDetailPage.jsx	Información detallada de una clase
DesignsPage.jsx	Gestión de diagramas creados
AccountPage.jsx	Configuración y perfil del usuario
SuperAdminPage.jsx	Panel de administración
NotFoundPage.jsx	Página de error 404

Función de las páginas

Cada página fue diseñada para manejar una responsabilidad específica dentro del sistema.

HomePage.jsx

Funciona como la página principal de bienvenida, mostrando accesos rápidos e información general del sistema.

LoginPage.jsx y SignupPage.jsx

Administran todo el sistema de autenticación visual del usuario mediante formularios interactivos conectados con el backend.

EditorPage.jsx

Es una de las páginas más importantes del sistema, ya que contiene el editor visual de diagramas y todas sus herramientas interactivas.

ClassesPage.jsx y ClassDetailPage.jsx

Permiten gestionar las clases creadas por los usuarios, visualizar miembros, comentarios y diagramas asociados.

DesignsPage.jsx

Muestra todos los diagramas almacenados y permite administrarlos.

AccountPage.jsx

Permite visualizar y modificar información relacionada con la cuenta del usuario.

SuperAdminPage.jsx

Incluye herramientas administrativas avanzadas para la gestión general del sistema.

NotFoundPage.jsx

Maneja errores de navegación cuando una ruta no existe.

Components

El sistema utiliza componentes reutilizables para dividir funcionalidades independientes dentro de la interfaz.

Los componentes permiten reutilizar código visual y lógico en distintas páginas, evitando duplicación y facilitando el mantenimiento.

Componentes principales

Componente	Función
EditorUI.jsx	Estructura general del editor
EditorToolbar.jsx	Barra de herramientas del editor
EditorSidebar.jsx	Panel lateral de herramientas
DiagramShapes.jsx	Manejo de figuras y formas
RenderShape.jsx	Renderizado dinámico de figuras
Footer.jsx	Pie de página del sistema
PasswordInput.jsx	Campo reutilizable de contraseña
SuperAdminSidebar.jsx	Navegación del panel administrador

Función de los componentes

EditorUI.jsx

Controla la estructura visual general del editor de diagramas.

EditorToolbar.jsx

Contiene herramientas para:

- Crear figuras.
- Editar elementos.
- Exportar diagramas.
- Controlar acciones del editor.

EditorSidebar.jsx

Muestra paneles laterales con configuraciones y herramientas adicionales.

DiagramShapes.jsx

Define y administra las distintas formas disponibles dentro del editor.

RenderShape.jsx

Se encarga del renderizado dinámico de cada figura creada por el usuario.

Footer.jsx

Componente reutilizable para el pie de página global del sistema.

PasswordInput.jsx

Permite reutilizar campos seguros de contraseña en login y registro.

SuperAdminSidebar.jsx

Administra la navegación interna del panel de administración.

Ventajas de la arquitectura basada en componentes

La división modular utilizada aporta múltiples beneficios:

- Reutilización de código.
- Menor duplicación.
- Mayor organización.
- Escalabilidad.
- Facilidad de mantenimiento.
- Separación de responsabilidades.
- Desarrollo colaborativo más ordenado.

Gracias a esta arquitectura, el sistema puede seguir creciendo sin necesidad de reorganizar completamente el proyecto.

Context API

El proyecto implementa Context API mediante AuthContext.jsx para manejar el estado global de autenticación.

Esto permite compartir información entre múltiples componentes sin necesidad de enviar propiedades manualmente entre niveles de componentes.

Funciones principales de AuthContext

- Mantener la sesión iniciada.
- Almacenar información del usuario.
- Compartir datos globales.
- Controlar accesos.
- Administrar tokens JWT.
- Manejar cierre de sesión.

Beneficios del Context API

La implementación del Context API permite:

- Reducir complejidad.
- Evitar prop drilling.
- Centralizar autenticación.
- Mejorar organización del estado global.
- Facilitar el control de permisos.

API Layer

El frontend se comunica con el backend mediante una capa de servicios API centralizada.

Los archivos principales encontrados son:

Archivo	Función
api.js	Configuración general de Axios
authApi.js	Solicitudes relacionadas con autenticación

Funcionamiento de la capa API

La API Layer permite centralizar todas las solicitudes HTTP realizadas al backend.

Las funciones principales incluyen:

- Login.
- Registro.
- Obtención de datos.
- Envío de información.
- Actualización de contenido.
- Eliminación de registros.

Todas las solicitudes utilizan Axios y trabajan mediante APIs REST.

Ventajas de centralizar las APIs

Centralizar las peticiones HTTP permite:

- Mejor organización.
- Código reutilizable.
- Mayor mantenimiento.
- Separación entre interfaz y lógica.
- Facilidad para modificar endpoints.
- Mejor manejo de errores.

Flujo general de la arquitectura Frontend

El funcionamiento general de la arquitectura es el siguiente:

1. El usuario interactúa con una Page.
2. La página utiliza Components reutilizables.
3. Los Components obtienen información desde Context API o APIs.
4. Axios envía solicitudes al backend.
5. El backend responde con datos JSON.
6. React actualiza dinámicamente la interfaz.

Este flujo permite mantener una aplicación rápida, organizada y completamente dinámica.

División de tareas

El desarrollo del frontend de DiagramTec fue organizado mediante una división de tareas por sectores funcionales. Esta metodología permitió distribuir el trabajo de forma más eficiente, mejorar la organización del equipo y facilitar el desarrollo simultáneo de múltiples partes del sistema.

Cada sector del frontend se encargó de una responsabilidad específica dentro de la plataforma, permitiendo mantener una estructura ordenada y escalable.

La división principal de tareas se organizó en:

- Diseño visual
- Navegación
- Editor de diagramas
- Autenticación
- Administración
- Integración con backend

Diseño visual

El sector de diseño visual fue responsable de toda la apariencia gráfica de la plataforma y de la experiencia de usuario.

Su principal objetivo fue desarrollar una interfaz moderna, clara y sencilla de utilizar.

Responsabilidades principales

- Diseño de interfaz.
- Distribución visual de elementos.
- Organización de paneles.
- Diseño responsive.
- Estilos CSS.
- Experiencia de usuario (UX).
- Adaptación visual entre páginas.
- Creación de formularios interactivos.
- Diseño de botones y menús.

Trabajo realizado

Se desarrollaron estilos personalizados utilizando CSS3 para mantener una identidad visual uniforme en todo el sistema.

También se trabajó en:

- Alineación de componentes.
- Organización de barras laterales.
- Diseño del editor.
- Animaciones visuales.
- Colores y tipografías.
- Distribución adaptable para distintas resoluciones.

El diseño buscó mejorar la facilidad de uso y reducir la complejidad visual del sistema.

Navegación

El sector de navegación fue responsable de administrar el desplazamiento entre páginas y controlar el flujo general de la aplicación.

La navegación fue implementada utilizando React Router DOM.

Responsabilidades principales

- Configuración de rutas.
- Navegación dinámica.
- Protección de rutas privadas.
- Redirecciones automáticas.
- Manejo de páginas inexistentes.
- Organización de vistas principales.

Trabajo realizado

Se configuró una estructura de rutas organizada para separar correctamente cada sección de la plataforma.

Entre las páginas administradas se encuentran:

- HomePage
- LoginPage
- SignupPage
- EditorPage
- ClassesPage
- AccountPage
- SuperAdminPage

También se implementó:

- Página 404.
- Redirección automática según autenticación.
- Protección de contenido privado.
- Control de acceso según sesión.

Esto permitió que la aplicación funcione como una SPA (Single Page Application), evitando recargas innecesarias.

Editor de diagramas

El editor de diagramas fue uno de los sectores más complejos e importantes del frontend.

Este módulo fue desarrollado para permitir la creación visual e interactiva de diagramas directamente desde el navegador.

Responsabilidades principales

- Creación visual de diagramas.
- Renderizado dinámico.
- Manipulación de figuras.
- Organización de elementos.
- Herramientas del editor.
- Exportación de contenido.
- Actualización en tiempo real.

Trabajo realizado

Se desarrollaron múltiples componentes específicos para el funcionamiento del editor:

Componente	Función
EditorUI.jsx	Estructura general
EditorToolbar.jsx	Herramientas del editor
EditorSidebar.jsx	Panel lateral
DiagramShapes.jsx	Administración de figuras
RenderShape.jsx	Renderizado dinámico

Funcionalidades implementadas

El editor permite:

- Crear figuras.
- Mover elementos.
- Editar contenido.
- Organizar diagramas.
- Guardar cambios.
- Exportar diagramas como imagen o PDF.

El renderizado dinámico permite actualizar visualmente el contenido sin recargar la página.

Autenticación

El sector de autenticación fue responsable de gestionar el acceso seguro al sistema.

Se desarrolló un sistema completo de autenticación conectado con el backend mediante APIs REST.

Responsabilidades principales

- Login.
- Registro de usuarios.
- Persistencia de sesión.
- Manejo de tokens JWT.
- Protección de rutas.
- Control de permisos.
- Cierre de sesión.

Trabajo realizado

Se implementaron formularios interactivos para login y registro utilizando React.

También se utilizó Context API mediante AuthContext.jsx para:

- Mantener usuarios autenticados.
- Compartir información global.
- Verificar permisos.
- Controlar accesos privados.

Además, Axios fue utilizado para enviar solicitudes de autenticación al backend.

Administración

El sector de administración fue desarrollado para permitir el control general de la plataforma mediante herramientas exclusivas para administradores.

Responsabilidades principales

- Panel administrador.
- Gestión de usuarios.
- Gestión de clases.
- Administración de contenido.
- Control del sistema.

Trabajo realizado

Se desarrolló una interfaz administrativa separada del resto del sistema.

El panel administrador permite:

- Visualizar usuarios.
- Administrar clases.
- Gestionar diagramas.
- Supervisar contenido general.

También se implementaron componentes exclusivos como:

- SuperAdminPage.jsx
- SuperAdminSidebar.jsx

Integración con Backend

Otro sector importante fue la conexión constante entre frontend y backend.

La comunicación fue desarrollada utilizando Axios y APIs REST.

Responsabilidades principales

- Envío de solicitudes HTTP.
- Recepción de datos JSON.
- Manejo de respuestas.
- Actualización dinámica de información.
- Validación de sesiones.

Trabajo realizado

Se desarrollaron archivos específicos como:

Archivo	Función
api.js	Configuración principal de Axios
authApi.js	Solicitudes de autenticación

La integración permitió:

- Obtener información desde MySQL.
- Guardar diagramas.
- Gestionar usuarios.
- Validar sesiones.
- Actualizar contenido en tiempo real.

Tecnologías utilizadas

Tecnología	Función
React 18	Librería principal del frontend
Vite	Entorno de desarrollo y compilación
React Router DOM	Navegación entre páginas
Axios	Comunicación con backend
jsPDF	Generación de archivos PDF
html2canvas	Captura visual de elementos HTML
dom-to-image-more	Conversión de diagramas a imagen
CSS3	Diseño visual y estilos
JavaScript	Lógica general del sistema

Importancia de la división de tareas

La división de tareas permitió:

- Mejor organización del desarrollo.
- Trabajo simultáneo entre sectores.
- Reducción de errores.
- Mayor escalabilidad.
- Separación de responsabilidades.
- Mantenimiento más sencillo.

Gracias a esta metodología, el frontend de DiagramTec pudo desarrollarse de forma más eficiente y ordenada, manteniendo una arquitectura moderna y fácil de ampliar en futuras versiones.

Sector Backend

Información general del sector

El backend de DiagramTec fue desarrollado utilizando Node.js junto con Express.js, permitiendo construir una API REST moderna, organizada y escalable. Este sector representa el núcleo lógico del sistema, ya que es el encargado de procesar toda la información enviada desde el frontend y gestionar la comunicación con la base de datos.

El backend fue diseñado para administrar todas las operaciones internas del sistema, garantizando el correcto funcionamiento de la plataforma y la seguridad de los datos almacenados.

Funciones principales del backend

El backend se encarga de:

- Procesamiento de datos.
- Gestión de usuarios.
- Autenticación.
- Manejo de clases.
- Administración de diagramas.
- Gestión de comentarios.
- Validación de información.
- Seguridad del sistema.
- Comunicación con MySQL.
- Integración entre Node.js y PHP.

Funcionamiento general

El sistema funciona mediante una arquitectura cliente-servidor.

El frontend desarrollado en React realiza solicitudes HTTP hacia la API creada con Express. El backend procesa estas solicitudes, ejecuta la lógica correspondiente y devuelve respuestas en formato JSON.

El flujo general es:

1. El usuario realiza una acción desde la interfaz.
2. React envía una solicitud HTTP mediante Axios.
3. Express recibe la petición.
4. El backend procesa la lógica necesaria.
5. Se realizan consultas a MySQL.
6. El servidor devuelve una respuesta JSON.
7. React actualiza la interfaz dinámicamente.

API REST

El backend fue desarrollado siguiendo el modelo API REST, utilizando distintos endpoints para separar las funcionalidades del sistema.

Cada endpoint se encarga de manejar una operación específica:

- Login.
- Registro.
- Creación de clases.
- Gestión de diagramas.
- Comentarios.
- Administración.
- Exportaciones.
- Gestión de usuarios.

Las respuestas son enviadas en formato JSON para facilitar la integración con el frontend.

Arquitectura utilizada

El backend utiliza una arquitectura organizada en capas (Layered Architecture), permitiendo separar responsabilidades y mantener un código más limpio, modular y escalable.

La estructura principal del backend se divide en:

- Controllers
- Services
- Repositories
- Routes
- Middlewares

Cada capa posee responsabilidades específicas dentro del sistema.

Controllers

Los Controllers son responsables de recibir las solicitudes HTTP provenientes del frontend y coordinar el flujo de ejecución.

Funcionan como intermediarios entre las rutas y la lógica de negocio.

Archivos Principales

Archivo	Función
authController.js	Control de autenticación
classesController.js	Gestión de clases
commentsController.js	Gestión de comentarios
designsController.js	Gestión de diagramas
adminController.js	Funciones administrativas
phpController.js	Comunicación con PHP

Funciones principales de los Controllers

- Recibir requests HTTP.
- Leer parámetros y datos enviados.
- Validar información inicial.
- Llamar Services.
- Procesar respuestas.
- Retornar JSON al frontend.
- Manejar códigos de estado HTTP.

Funcionamiento de los Controllers

Cuando el frontend realiza una solicitud:

1. La Route correspondiente recibe la petición.
2. El Controller asociado procesa la solicitud.
3. Se validan datos básicos.
4. Se llama al Service correspondiente.
5. El resultado se devuelve al cliente.

Esto permite mantener separada la lógica de negocio de la lógica de rutas.

Services

La capa Services contiene toda la lógica principal del sistema.

Es una de las partes más importantes del backend, ya que aquí se procesan las reglas de negocio y las operaciones centrales de la plataforma.

Archivos principales

Archivo	Función
authService.js	Lógica de autenticación
classService.js	Gestión lógica de clases
commentService.js	Procesamiento de comentarios
designService.js	Gestión de diagramas
adminService.js	Funciones administrativas

Funciones principales de los Services

- Procesamiento de información.
- Validaciones avanzadas.
- Reglas del sistema.
- Verificación de permisos.
- Manipulación de datos.
- Coordinación entre repositories.
- Operaciones complejas.

Trabajo realizado en Services

Dentro de esta capa se implementaron funcionalidades como:

- Verificación de usuarios.
- Comparación de contraseñas.
- Generación de JWT.
- Creación de diagramas.
- Validación de clases.
- Administración de permisos.
- Manejo de copias de diagramas.
- Gestión de relaciones entre tablas.

Esta separación mejora enormemente el mantenimiento y la reutilización del código.

Repositories

La capa Repository se encarga del acceso directo a la base de datos.

Aquí se encuentran todas las consultas SQL necesarias para interactuar con MySQL.

Archivos principales

Archivo	Función
userRepository.js	Operaciones sobre usuarios
classRepository.js	Operaciones sobre clases
commentRepository.js	Operaciones sobre comentarios
designRepository.js	Operaciones sobre diagramas
adminRepository.js	Operaciones administrativas

Funciones principales de los Repositories

- Queries SQL.
- Inserciones.
- Actualizaciones.
- Eliminaciones.
- Búsquedas.
- Consultas relacionales.
- Obtención de registros.

Ventajas de utilizar Repositories

La implementación de repositories permite:

- Separar SQL de la lógica del sistema.
- Reutilizar consultas.
- Mejor mantenimiento.
- Código más organizado.
- Mayor escalabilidad.

Además, facilita futuras modificaciones en la base de datos.

Routes

Las Routes son responsables de definir todos los endpoints de la API REST.

Cada ruta conecta una URL específica con un Controller determinado.

Archivos principales

Archivo	Función
auth.js	Rutas de autenticación
classes.js	Rutas de clases
comments.js	Rutas de comentarios
designs.js	Rutas de diagramas
admin.js	Rutas administrativas
phpProxy.js	Rutas de conexión con PHP

Funciones principales de las Routes

- Definir endpoints.
- Asociar Controllers.
- Organizar URLs.
- Aplicar middlewares.
- Controlar accesos.

Ejemplos de endpoints utilizados

Método	Endpoint	Función
POST	/auth/login	Inicio de sesión
POST	/auth/signup	Registro
GET	/classes	Obtener clases
POST	/designs	Crear diseño
GET	/comments	Obtener comentarios

Middlewares

Los Middlewares son funciones intermedias que se ejecutan antes de llegar al Controller.

Se utilizan para controlar seguridad, errores y validaciones.

Archivos principales

Archivo	Función
auth.js	Verificación de autenticación
asyncHandler.js	Manejo de errores asíncronos
errorHandler.js	Control global de errores

Funciones principales de los Middlewares

- Verificación de tokens JWT.
- Protección de rutas privadas.
- Validación de accesos.
- Manejo centralizado de errores.
- Captura de excepciones.
- Control de permisos.

Middleware de autenticación

El middleware auth.js verifica:

- Existencia de token.
- Validez del JWT.
- Identidad del usuario.
- Permisos de acceso.

Gracias a esto, ciertas rutas solo pueden ser utilizadas por usuarios autenticados.

Manejo de errores

El sistema implementa middlewares especializados para evitar fallos inesperados.

Esto permite:

- Evitar caídas del servidor.
- Enviar mensajes claros.
- Centralizar errores.
- Mejorar la seguridad.

Beneficios de la arquitectura Backend

La arquitectura implementada aporta múltiples ventajas:

- Separación de responsabilidades.
- Código modular.
- Escalabilidad.
- Facilidad de mantenimiento.
- Reutilización de lógica.
- Mejor organización.
- Mayor seguridad.
- Desarrollo colaborativo más eficiente.

Tecnologías utilizadas

El backend de DiagramTec fue desarrollado utilizando distintas tecnologías modernas orientadas al desarrollo de aplicaciones web escalables, seguras y organizadas. Cada herramienta utilizada cumple una función específica dentro de la arquitectura general del sistema.

La combinación de estas tecnologías permitió construir un backend robusto, capaz de manejar autenticación, procesamiento de datos, conexión con bases de datos y comunicación entre distintos servicios.

Tecnologías principales utilizadas

Tecnología	Función
Node.js	Entorno de ejecución del servidor
Express.js	Framework backend para APIs REST
MySQL2	Conexión y consultas a la base de datos
JWT	Sistema de autenticación mediante tokens
bcryptjs	Encriptación segura de contraseñas
dotenv	Manejo de variables de entorno
CORS	Control de comunicación segura
Axios	Comunicación HTTP entre servicios
PHP	Servicios complementarios e integración híbrida

Node.js

Node.js fue utilizado como entorno principal de ejecución del backend.

Gracias a Node.js, el sistema puede ejecutar JavaScript del lado del servidor y manejar múltiples solicitudes simultáneamente utilizando programación asíncrona.

Funciones de Node.js dentro del proyecto

- Ejecución del servidor backend.
- Procesamiento de solicitudes HTTP.
- Comunicación con MySQL.
- Manejo de APIs REST.
- Integración entre servicios.
- Gestión de autenticación.
- Procesamiento de datos en tiempo real.

Ventajas de utilizar Node.js

- Alta velocidad de ejecución.
- Arquitectura asincrónica.
- Escalabilidad.
- Compatibilidad completa con JavaScript.
- Gran ecosistema de librerías.

Express.js

Express.js fue utilizado como framework principal del backend para construir la API REST.

Express simplifica enormemente la creación de rutas, middlewares y controladores.

Funciones de Express.js

- Creación de endpoints.
- Gestión de rutas.
- Manejo de middlewares.
- Procesamiento de requests y responses.
- Organización de la API.
- Comunicación cliente-servidor.

Implementación dentro del proyecto

Express fue utilizado para:

- Rutas de autenticación.
- Gestión de diagramas.
- Administración de clases.
- Manejo de comentarios.
- Integración con PHP.
- Protección de endpoints.

MySQL2

La librería MySQL2 fue utilizada para conectar Node.js con la base de datos MySQL.

Permite ejecutar consultas SQL de manera rápida y eficiente.

Funciones de MySQL2

- Conexión con MySQL.
- Ejecución de queries.
- Inserción de datos.
- Actualización de registros.
- Eliminación de información.
- Consultas relacionales.

Uso dentro del proyecto

La librería fue utilizada principalmente en:

- Repositories.
- Consultas SQL.
- Gestión de usuarios.
- Diagramas.
- Comentarios.
- Relaciones entre clases y usuarios.

JWT (JSON Web Token)

JWT fue implementado como sistema de autenticación principal.

Los tokens permiten verificar la identidad de los usuarios sin almacenar sesiones tradicionales en el servidor.

Funciones de JWT

- Autenticación de usuarios.
- Protección de rutas.
- Persistencia de sesión.
- Validación de identidad.
- Control de accesos.

Funcionamiento dentro del sistema

1. El usuario inicia sesión.
2. El backend valida las credenciales.
3. Se genera un token JWT.
4. El frontend almacena el token.
5. Cada solicitud protegida envía el JWT.
6. El middleware verifica el token.

Esto permite mantener sesiones seguras y eficientes.

bcryptjs

bcryptjs fue utilizado para encriptar las contraseñas de los usuarios antes de almacenarlas en la base de datos.

Esto mejora significativamente la seguridad del sistema.

Funciones de bcryptjs

- Hash de contraseñas.
- Comparación segura de credenciales.
- Protección contra robo de contraseñas.

Funcionamiento

Cuando un usuario se registra:

1. La contraseña se encripta.
2. Se almacena el hash en MySQL.
3. Nunca se guarda la contraseña original.

Durante el login:

1. bcrypt compara el hash almacenado.
2. Verifica si coincide con la contraseña ingresada.

---

dotenv

dotenv fue utilizado para manejar variables de entorno y evitar exponer información sensible dentro del código fuente.

Información almacenada mediante dotenv

- Claves JWT.
- Datos de conexión MySQL.
- URLs del servidor.
- Configuraciones privadas.

Beneficios

- Mayor seguridad.
- Configuración centralizada.
- Protección de credenciales.
- Fácil mantenimiento.

CORS

CORS (Cross-Origin Resource Sharing) fue implementado para controlar qué dominios pueden comunicarse con el backend.

Funciones de CORS

- Permitir solicitudes seguras.
- Evitar accesos externos no autorizados.
- Controlar comunicación entre frontend y backend.

Uso dentro del proyecto

El frontend y backend funcionan en puertos distintos durante el desarrollo, por lo que CORS fue necesario para habilitar la comunicación entre ambos servicios.

Axios

Axios fue utilizado para realizar solicitudes HTTP entre distintos sectores del sistema.

Aunque es más utilizado en frontend, también puede utilizarse dentro del backend para comunicación entre servicios externos.

Funciones de Axios

- Envío de requests HTTP.
- Comunicación con APIs.
- Integración Node.js ↔ PHP.
- Manejo de respuestas JSON.

Uso dentro del proyecto

Axios fue utilizado especialmente en:

- Comunicación frontend-backend.
- Integración híbrida con PHP.
- Solicitudes internas entre servicios.

PHP

El proyecto incorpora PHP como complemento del backend principal desarrollado en Node.js.

Esto permitió integrar funcionalidades específicas sin modificar la arquitectura principal del sistema.

Funciones de PHP dentro del proyecto

- Servicios auxiliares.
- Compatibilidad con scripts externos.
- Procesamiento complementario.
- Integración híbrida.

Beneficios de utilizar PHP

- Compatibilidad con herramientas existentes.
- Integración de servicios heredados.
- Flexibilidad arquitectónica.

Cómo fue la conexión entre Node.js y PHP

DiagramTec implementa una arquitectura híbrida donde Node.js funciona como backend principal mientras que PHP actúa como servicio complementario para determinadas funcionalidades.

Esta integración permite combinar tecnologías diferentes dentro de una misma plataforma sin afectar el funcionamiento general del sistema.

Funcionamiento general de la integración

La conexión entre Node.js y PHP se realiza mediante Express actuando como intermediario entre el frontend y los scripts PHP.

El backend recibe solicitudes desde React y decide cuándo redireccionar determinadas operaciones hacia PHP.

Archivos involucrados

Archivo	Función
phpController.js	Controlador de comunicación con PHP
phpProxy.js	Proxy de rutas hacia servicios PHP

Flujo de funcionamiento

El funcionamiento de la integración sigue el siguiente proceso:

1. El frontend realiza una solicitud HTTP.
2. Express recibe la solicitud.
3. La ruta correspondiente ejecuta phpController.js.
4. El controlador redirecciona la petición hacia PHP.
5. PHP procesa la operación necesaria.
6. PHP devuelve una respuesta.
7. Node.js recibe la información.
8. Express responde al frontend en formato JSON.

Objetivo de la integración híbrida

La integración entre Node.js y PHP fue implementada para:

- Aprovechar funcionalidades específicas.
- Mantener compatibilidad con servicios externos.
- Integrar herramientas heredadas.
- Evitar reescribir sistemas completos.
- Centralizar el acceso desde Express.

Beneficios de la conexión Node.js + PHP

La arquitectura híbrida aporta múltiples ventajas:

- Flexibilidad tecnológica.
- Integración de distintos servicios.
- Compatibilidad con sistemas externos.
- Mejor organización de funcionalidades.
- Separación de responsabilidades.
- Escalabilidad futura.

Sector de Documentación

El sector de documentación de DiagramTec fue el encargado de desarrollar, organizar y estructurar toda la información técnica y descriptiva del proyecto. Su función principal fue registrar de manera detallada el funcionamiento del sistema, las tecnologías utilizadas, la arquitectura implementada y los distintos procesos realizados durante el desarrollo.

La documentación cumple un rol fundamental dentro del proyecto, ya que permite comprender la estructura general de DiagramTec, facilitar el mantenimiento futuro y servir como guía técnica tanto para desarrolladores como para evaluadores del sistema.

Además, este sector permitió centralizar toda la información relacionada con frontend, backend, base de datos, seguridad, diagramas y funcionamiento interno de la plataforma.

Objetivos del sector de documentación

El sector fue desarrollado con múltiples objetivos principales:

- Explicar el funcionamiento general del sistema.
- Documentar la arquitectura utilizada.
- Registrar tecnologías implementadas.
- Organizar la información técnica.
- Facilitar el mantenimiento futuro.
- Mejorar la comprensión del proyecto.
- Servir como guía para desarrolladores.
- Presentar formalmente el sistema.

Trabajo realizado dentro del sector

El sector de documentación fue responsable de desarrollar una gran cantidad de contenido técnico y organizativo relacionado con DiagramTec.

Entre los principales trabajos realizados se encuentran:

- Documentación del frontend.
- Documentación del backend.
- Explicación de arquitectura.
- Desarrollo del diccionario de datos.
- Relaciones de base de datos.
- Diagramas del sistema.
- Explicación del flujo web.
- Seguridad implementada.
- Funcionamiento interno.
- Organización del proyecto.
- Explicación de APIs REST.
- Documentación de autenticación JWT.
- Explicación de tecnologías utilizadas.

Documentación del Frontend

Dentro de la documentación frontend se desarrollaron apartados relacionados con:

- Arquitectura modular.
- React.
- Vite.
- Componentes reutilizables.
- Navegación.
- Context API.
- Editor de diagramas.
- Exportación de contenido.
- Integración con backend.

También se documentó la organización de carpetas y el funcionamiento general de la interfaz visual.

Documentación del Backend

El sector documentó completamente la arquitectura backend desarrollada con Node.js y Express.

Se explicó:

- Arquitectura en capas.
- Controllers.
- Services.
- Repositories.
- Routes.
- Middlewares.
- APIs REST.
- Seguridad.
- Validaciones.
- Integración Node.js + PHP.

Esto permitió describir detalladamente cómo funciona el núcleo lógico del sistema.

Documentación de la base de datos

Una de las tareas más importantes realizadas fue la documentación completa de la base de datos relacional.

Trabajo realizado

Se desarrolló:

- Explicación de tablas.
- Relaciones entre entidades.
- Integridad referencial.
- Restricciones SQL.
- Índices.
- Claves primarias.
- Claves foráneas.
- Relaciones muchos a muchos.
- Relaciones recursivas.

Tablas documentadas

Tabla	Función
users	Usuarios del sistema
classes	Clases virtuales
class_members	Relación usuarios-clases
designs	Diagramas
comments	Comentarios

Diccionario de datos

El sector de documentación desarrolló un diccionario de datos completo para explicar la estructura de la base de datos.

El diccionario incluye:

- Nombre de tablas.
- Campos.
- Tipos de datos.
- Restricciones.
- Claves primarias.
- Claves foráneas.
- Descripción funcional de cada atributo.

Diagramas del sistema

También se realizaron distintos diagramas técnicos para representar visualmente la estructura y funcionamiento de DiagramTec.

Diagramas desarrollados

- Diagrama de entidad-relación.
- Diagramas de relaciones.
- Diagramas de arquitectura.
- Diagramas de flujo.
- Diagramas de casos de uso.

Objetivo de los diagramas

Los diagramas permiten:

- Visualizar relaciones del sistema.
- Comprender el flujo interno.
- Representar la arquitectura.
- Facilitar la explicación técnica.
- Mejorar la interpretación del proyecto.

Documentación del flujo web

El sector también documentó el funcionamiento general de la aplicación web.

Se explicó detalladamente:

- Registro de usuarios.
- Inicio de sesión.
- Creación de diagramas.
- Gestión de clases.
- Exportación PDF.
- Comunicación cliente-servidor.
- Flujo de autenticación JWT.

Documentación de seguridad

La documentación incluyó todos los mecanismos de seguridad implementados dentro del sistema.

Temas documentados

- Encriptación de contraseñas.
- bcryptjs.
- JWT.
- Middleware de autenticación.
- Validaciones backend.
- CORS.
- Restricciones SQL.
- Protección de rutas.

Herramientas utilizadas

Para desarrollar la documentación se utilizaron distintas herramientas técnicas y de organización.

Herramientas principales

Herramienta	Función
Microsoft Word	Redacción del informe
Draw.io	Diagramas y esquemas
MySQL Workbench	Modelado de base de datos
Visual Studio Code	Análisis del código
GitHub	Organización del proyecto
XAMPP	Pruebas locales
ChatGPT	Asistencia técnica y redacción

Organización del sector

El sector de documentación se organizó mediante división de tareas para facilitar la creación del informe técnico.

Tareas realizadas

- Redacción técnica.
- Organización del contenido.
- Desarrollo de tablas.
- Diagramas visuales.
- Revisión de arquitectura.
- Explicación de procesos.
- Documentación de tecnologías.
- Análisis de código.

Importancia de la documentación

La documentación es una de las partes más importantes del proyecto debido a que permite:

- Comprender el sistema.
- Facilitar futuras modificaciones.
- Mantener organización.
- Explicar decisiones técnicas.
- Ayudar en mantenimiento.
- Mejorar escalabilidad.

Además, permite que otros desarrolladores puedan comprender DiagramTec de manera más sencilla.

Relación con otros sectores

El sector de documentación trabajó conjuntamente con:

Sector	Relación
Frontend	Documentación visual y arquitectura
Backend	Explicación técnica y APIs
Base de datos	Relaciones y estructura SQL
Marketing	Organización visual del informe

Beneficios obtenidos

- Mayor organización.
- Explicación detallada del sistema.
- Facilidad de mantenimiento.
- Comprensión técnica completa.
- Presentación profesional del proyecto.
- Centralización de información.

La base de datos utilizada en DiagramTec es MySQL, un sistema gestor de bases de datos relacional ampliamente utilizado en aplicaciones web modernas debido a su rendimiento, estabilidad y capacidad para manejar relaciones complejas entre datos.

La base de datos cumple un rol fundamental dentro del sistema, ya que es la encargada de almacenar toda la información relacionada con usuarios, clases, diagramas, comentarios y permisos.

Conexión con la base de datos

La conexión entre el backend y MySQL se realiza mediante la librería mysql2.

Dentro del proyecto existe un archivo principal encargado de establecer esta conexión:

- db.js

Este archivo administra:

- Configuración del servidor MySQL.
- Usuario y contraseña de acceso.
- Nombre de la base de datos.
- Inicialización de conexiones.
- Exportación de la conexión para el resto del sistema.

Funcionamiento de db.js

El archivo db.js actúa como punto central de acceso a la base de datos.

Cuando el servidor backend inicia:

1. Node.js carga la configuración.
2. mysql2 establece la conexión con MySQL.
3. La conexión queda disponible para repositories y services.
4. El sistema puede ejecutar consultas SQL.

Esto permite que todos los módulos del backend puedan interactuar con la base de datos de forma organizada y reutilizable.

Estructura general de la base de datos

La base de datos fue diseñada utilizando un modelo relacional compuesto por múltiples tablas conectadas mediante claves foráneas.

Las tablas principales del sistema son:

- users
- classes
- class_members
- designs
- comments

Cada tabla cumple una función específica dentro del funcionamiento de DiagramTec.

Tabla users

La tabla users almacena toda la información relacionada con los usuarios registrados dentro del sistema.

Información almacenada

- Nombre del usuario.
- Apellido.
- Correo electrónico.
- Contraseña encriptada.
- Rol del usuario.
- Fecha de creación.

Función dentro del sistema

Esta tabla permite:

- Registrar usuarios.
- Gestionar autenticación.
- Administrar permisos.
- Identificar propietarios de clases y diagramas.

Además, funciona como tabla principal para múltiples relaciones con otras entidades del sistema.

Tabla classes

La tabla classes almacena las clases creadas dentro de la plataforma.

Cada clase pertenece a un usuario que actúa como propietario o profesor.

Información almacenada

- Título de la clase.
- Descripción.
- Código de acceso.
- Usuario creador.
- Fecha de creación.

Función dentro del sistema

Esta tabla permite:

- Crear espacios académicos.
- Organizar diagramas.
- Gestionar estudiantes.
- Compartir contenido mediante códigos de acceso.

Tabla class_members

La tabla class_members funciona como tabla intermedia para resolver la relación muchos a muchos entre usuarios y clases.

Relación implementada

Un usuario puede pertenecer a múltiples clases y una clase puede contener múltiples usuarios.

Para resolver esta relación se utiliza class_members.

Información almacenada

- Usuario asociado.
- Clase asociada.
- Fecha de ingreso.

Función dentro del sistema

Permite:

- Registrar estudiantes en clases.
- Controlar miembros.
- Gestionar participación.
- Relacionar usuarios y clases dinámicamente.

Tabla designs

La tabla designs almacena todos los diagramas creados por los usuarios.

Es una de las tablas más importantes del sistema debido a que concentra gran parte de la información principal de DiagramTec.

Información almacenada

- Título del diagrama.
- Contenido.
- Imagen generada.
- Archivo PDF.
- Descripción.
- Usuario propietario.
- Clase asociada.
- Información de copias.
- Fecha de creación.

Función dentro del sistema

Permite:

- Guardar diagramas.
- Recuperar diseños.
- Exportar contenido.
- Compartir trabajos.
- Crear copias de diagramas.

Además, incluye una autorrelación mediante originalId para gestionar diagramas duplicados o derivados.

Tabla comments

La tabla comments almacena los comentarios realizados por los usuarios dentro de las clases.

Información almacenada

- Usuario autor.
- Clase relacionada.
- Contenido del comentario.
- Fecha de publicación.

Función dentro del sistema

Permite:

- Comunicación entre usuarios.
- Participación en clases.
- Interacción académica.
- Retroalimentación dentro de la plataforma.

Relaciones entre tablas

La base de datos fue diseñada utilizando relaciones relacionales mediante claves foráneas.

Estas relaciones permiten mantener conectada toda la información del sistema.

Relaciones principales

Relación	Tipo
users → classes	Uno a muchos
users → designs	Uno a muchos
users → comments	Uno a muchos
classes → designs	Uno a muchos
classes → comments	Uno a muchos
users ↔ classes	Muchos a muchos
designs → designs	Relación recursiva

Integridad referencial

El sistema utiliza claves foráneas (Foreign Keys) para mantener la integridad de los datos.

Esto garantiza que:

- No existan registros inválidos.
- Las relaciones permanezcan consistentes.
- Las eliminaciones se controlen correctamente.

Acciones implementadas

La base de datos utiliza:

- ON DELETE CASCADE
- ON DELETE SET NULL

Ejemplo de funcionamiento

Si un usuario es eliminado:

- Sus clases pueden eliminarse automáticamente.
- Sus diagramas asociados también pueden eliminarse.
- Los registros relacionados mantienen coherencia.

Esto evita datos huérfanos dentro de la base de datos.

Restricciones utilizadas

La base de datos implementa distintas restricciones para mejorar la seguridad y consistencia.

Restricciones implementadas

Restricción	Función
PRIMARY KEY	Identificación única
FOREIGN KEY	Relaciones entre tablas
UNIQUE	Evitar duplicados
NOT NULL	Campos obligatorios
AUTO_INCREMENT	IDs automáticos

Índices

El sistema implementa índices para mejorar el rendimiento de búsqueda y consultas frecuentes.

Índices implementados

- ownerId
- classId
- userId

Beneficios de los índices

- Consultas más rápidas.
- Mejor rendimiento.
- Optimización del sistema.
- Mayor eficiencia en relaciones.

Funcionamiento general de la base de datos

El funcionamiento general de la base de datos sigue el siguiente flujo:

1. El frontend solicita información.
2. Express recibe la petición.
3. Los repositories ejecutan consultas SQL.
4. MySQL procesa la información.
5. Los resultados vuelven al backend.
6. Express devuelve respuestas JSON.
7. React actualiza la interfaz.

Funcionamiento web

El funcionamiento general de DiagramTec se basa en una arquitectura cliente-servidor donde frontend, backend y base de datos trabajan conjuntamente.

La aplicación funciona completamente mediante comunicación HTTP utilizando APIs REST.

Flujo general del sistema web

El funcionamiento principal de la plataforma sigue el siguiente proceso:

1. El usuario accede al frontend desde el navegador.
2. React renderiza la interfaz visual.
3. El usuario interactúa con el sistema.
4. Axios envía solicitudes HTTP hacia Express.
5. El backend procesa la información.
6. Express consulta MySQL.
7. MySQL devuelve los resultados.
8. El backend responde en formato JSON.
9. React actualiza la interfaz dinámicamente.

Comunicación cliente-servidor

DiagramTec trabaja completamente bajo el modelo cliente-servidor.

Frontend (Cliente)

El frontend se encarga de:

- Mostrar la interfaz.
- Capturar acciones del usuario.
- Enviar solicitudes HTTP.
- Actualizar componentes visuales.

Backend (Servidor)

El backend se encarga de:

- Procesar lógica.
- Validar datos.
- Gestionar autenticación.
- Consultar MySQL.
- Aplicar seguridad.

Base de datos

MySQL se encarga de:

- Almacenar información.
- Gestionar relaciones.
- Ejecutar consultas.
- Mantener integridad referencial.

Flujo de autenticación

El sistema implementa autenticación basada en JWT.

Funcionamiento

1. El usuario inicia sesión.
2. React envía credenciales.
3. Express valida datos.
4. bcrypt compara contraseñas.
5. JWT genera un token.
6. El frontend almacena el token.
7. Las solicitudes futuras utilizan ese token.

Flujo de creación de diagramas

1. El usuario accede al editor.
2. React renderiza herramientas visuales.
3. El usuario crea figuras.
4. El contenido se almacena temporalmente.
5. Axios envía la información al backend.
6. Express procesa el diseño.
7. MySQL guarda el diagrama.
8. El sistema devuelve confirmación.

Flujo de clases y comentarios

Clases

1. El profesor crea una clase.
2. El backend genera el código.
3. Los estudiantes ingresan mediante el código.
4. class_members almacena la relación.

Comentarios

1. El usuario publica un comentario.
2. Express valida el contenido.
3. comments almacena la información.
4. React actualiza la vista dinámicamente.

Actualización dinámica del sistema

Gracias a React y Axios, DiagramTec funciona de manera dinámica sin recargar páginas constantemente.

Esto permite:

- Mejor experiencia de usuario.
- Mayor velocidad.
- Navegación fluida.
- Actualización inmediata de contenido.

Beneficios de la arquitectura web implementada

La arquitectura utilizada aporta múltiples ventajas:

- Separación entre frontend y backend.
- Mejor organización.
- Escalabilidad.
- Seguridad.
- Mantenimiento sencillo.
- Comunicación eficiente.
- Mayor rendimiento.

DiagramTec fue desarrollado utilizando una arquitectura dividida en tres sectores principales:

- Frontend
- Backend
- Base de datos

Cada sector cumple funciones específicas dentro del sistema y se comunica constantemente con los demás mediante APIs REST y consultas SQL.

Esta separación de responsabilidades permite mantener una estructura organizada, escalable y más fácil de mantener.

Interacción entre Frontend y Backend

La comunicación principal del sistema se realiza entre el frontend desarrollado en React y el backend desarrollado con Node.js y Express.

El frontend funciona como la capa visual de la aplicación, mientras que el backend procesa toda la lógica interna y administra la información.

La interacción entre ambos sectores se realiza mediante solicitudes HTTP utilizando Axios y respuestas en formato JSON.

Frontend

El frontend es el encargado de toda la interacción visual con el usuario.

Responsabilidades principales del Frontend

- Mostrar información visual.
- Renderizar componentes dinámicamente.
- Capturar acciones del usuario.
- Validar formularios básicos.
- Navegación entre páginas.
- Enviar solicitudes HTTP al backend.
- Mostrar respuestas obtenidas desde la API.

Función del frontend dentro del sistema

Cuando el usuario realiza una acción:

- Crear un diagrama.
- Iniciar sesión.
- Publicar comentarios.
- Crear clases.
- Exportar contenido.

El frontend captura dicha acción y envía una solicitud hacia el backend para procesar la operación correspondiente.

Backend

El backend funciona como núcleo lógico del sistema.

Es el encargado de procesar toda la información recibida desde el frontend y ejecutar las operaciones necesarias dentro de la plataforma.

Responsabilidades principales del Backend

- Procesar lógica del sistema.
- Validar información.
- Verificar permisos.
- Gestionar autenticación.
- Generar JWT.
- Acceder a MySQL.
- Aplicar seguridad.
- Controlar rutas protegidas.
- Procesar diagramas y clases.

Función del backend dentro del sistema

Cuando el backend recibe una solicitud:

1. Verifica permisos.
2. Valida datos.
3. Ejecuta lógica.
4. Consulta MySQL.
5. Procesa resultados.
6. Devuelve una respuesta JSON.

Esto permite centralizar toda la lógica del sistema en un único sector.

Base de datos

La base de datos MySQL funciona como sistema de almacenamiento persistente de la plataforma.

Toda la información importante del sistema se almacena dentro de las tablas relacionales.

Responsabilidades principales de la base de datos

- Almacenamiento persistente.
- Relaciones entre entidades.
- Gestión de usuarios.
- Consultas SQL.
- Integridad referencial.
- Optimización mediante índices.
- Restricciones de seguridad.

Función de MySQL dentro del sistema

MySQL almacena:

- Usuarios.
- Clases.
- Diagramas.
- Comentarios.
- Relaciones entre usuarios y clases.
- Tokens y permisos asociados.

El backend accede constantemente a esta información mediante repositories y consultas SQL.

Comunicación completa entre sectores

El flujo de interacción entre sectores funciona de la siguiente manera:

1. El usuario interactúa con React.
2. React envía solicitudes HTTP utilizando Axios.
3. Express recibe las peticiones.
4. Los Controllers procesan la solicitud.
5. Los Services ejecutan lógica.
6. Los Repositories consultan MySQL.
7. MySQL devuelve resultados.
8. Express genera respuestas JSON.
9. React actualiza la interfaz dinámicamente.

Beneficios de la separación por sectores

La división entre frontend, backend y base de datos aporta múltiples ventajas:

- Mayor organización.
- Escalabilidad.
- Mejor mantenimiento.
- Seguridad centralizada.
- Desarrollo colaborativo.
- Reutilización de código.
- Separación de responsabilidades.
- Mejor rendimiento.

Gracias a esta arquitectura, DiagramTec puede crecer y agregar nuevas funcionalidades sin afectar el funcionamiento general del sistema.

Flujo dentro de la web

El funcionamiento interno de DiagramTec sigue múltiples flujos de ejecución según la acción realizada por el usuario.

Cada proceso involucra interacción entre frontend, backend y base de datos.

Flujo de registro de usuarios

El registro de usuarios permite crear nuevas cuentas dentro del sistema.

Proceso de registro

1. El usuario completa el formulario de registro.
2. React captura los datos ingresados.
3. Axios envía la información al backend.
4. Express valida los datos.
5. bcryptjs encripta la contraseña.
6. MySQL almacena el usuario.
7. JWT genera un token de autenticación.
8. El frontend almacena la sesión.

Validaciones realizadas

Durante el registro se verifican:

- Campos obligatorios.
- Correos duplicados.
- Formato de datos.
- Seguridad de contraseña.

Flujo de inicio de sesión

El login permite autenticar usuarios registrados.

Proceso de autenticación

1. El usuario ingresa email y contraseña.
2. Axios envía las credenciales.
3. Express recibe la solicitud.
4. MySQL busca el usuario.
5. bcryptjs compara contraseñas.
6. JWT genera un token válido.
7. El backend devuelve el token.
8. React almacena la sesión activa.

Persistencia de sesión

El token JWT permite:

- Mantener la sesión iniciada.
- Acceder a rutas privadas.
- Verificar identidad del usuario.

Flujo de creación de diagramas

El editor visual permite crear diagramas dinámicamente desde el navegador.

Proceso de creación

1. El usuario accede al editor.
2. React renderiza herramientas visuales.
3. El usuario crea figuras.
4. Los componentes actualizan el estado.
5. Axios envía el contenido al backend.
6. Express procesa la información.
7. MySQL almacena el diagrama.
8. El sistema confirma el guardado.

Información almacenada

Los diagramas pueden almacenar:

- Contenido.
- Imágenes.
- PDFs.
- Copias.
- Descripciones.

Flujo de exportación

DiagramTec permite exportar diagramas como imagen o PDF.

Proceso de exportación

1. El frontend captura el contenido visual.
2. html2canvas convierte el HTML en imagen.
3. dom-to-image-more procesa el contenido gráfico.
4. jsPDF genera el archivo PDF.
5. El navegador descarga automáticamente el archivo.

Objetivo de la exportación

Permitir que los usuarios puedan:

- Compartir diagramas.
- Guardar proyectos.
- Descargar contenido académico.

Flujo de gestión de clases

El sistema permite crear y administrar clases virtuales.

Proceso de creación de clases

1. El profesor crea una clase.
2. Express genera un código único.
3. MySQL almacena la clase.
4. El sistema devuelve confirmación.

Proceso de ingreso a clases

1. El estudiante ingresa un código.
2. El backend verifica la clase.
3. class_members crea la relación.
4. El usuario queda asociado a la clase.

Funcionalidades adicionales

Dentro de las clases los usuarios pueden:

- Compartir diagramas.
- Publicar comentarios.
- Visualizar miembros.
- Interactuar académicamente.

Seguridad implementada

DiagramTec implementa múltiples mecanismos de seguridad para proteger usuarios, información y funcionamiento general del sistema.

La seguridad fue aplicada tanto en frontend como en backend y base de datos.

Encriptación de contraseñas

Las contraseñas nunca se almacenan en texto plano.

El sistema utiliza bcryptjs para generar hashes seguros antes de guardar información en MySQL.

Funcionamiento

1. El usuario ingresa una contraseña.
2. bcryptjs genera un hash encriptado.
3. MySQL almacena únicamente el hash.
4. Durante el login se comparan hashes.

Beneficios

- Protección contra robo de credenciales.
- Mayor seguridad.
- Prevención de filtraciones directas.

JWT (JSON Web Token)

La autenticación del sistema utiliza JWT.

Los tokens permiten mantener sesiones seguras sin almacenar sesiones tradicionales en el servidor.

Funciones de JWT

- Verificar usuarios.
- Mantener sesiones activas.
- Proteger endpoints.
- Validar permisos.
- Controlar accesos.

Funcionamiento

1. El usuario inicia sesión.
2. El backend genera un token.
3. React almacena el JWT.
4. Cada solicitud protegida envía el token.
5. El middleware verifica autenticidad.

Middleware de autenticación

El middleware auth.js protege rutas privadas del sistema.

Funciones principales

- Verificar tokens válidos.
- Validar identidad del usuario.
- Controlar permisos.
- Bloquear accesos no autorizados.

Rutas protegidas

El middleware protege funcionalidades como:

- Creación de diagramas.
- Administración.
- Gestión de clases.
- Acceso a información privada.

CORS

CORS fue implementado para controlar qué dominios pueden acceder al backend.

Objetivos de CORS

- Evitar accesos externos no autorizados.
- Permitir comunicación segura.
- Controlar solicitudes entre frontend y backend.

Validaciones implementadas

El backend valida constantemente la información recibida desde el frontend.

Validaciones realizadas

- Tipos de datos.
- Campos obligatorios.
- Permisos de usuario.
- Accesos privados.
- Tokens válidos.
- Existencia de registros.

Restricciones SQL

La base de datos implementa restricciones para proteger la integridad de la información.

Restricciones utilizadas

Restricción	Función
UNIQUE	Evitar duplicados
FOREIGN KEY	Mantener relaciones
CASCADE	Eliminación automática relacionada
SET NULL	Mantener referencias válidas

Integridad referencial

Las claves foráneas garantizan que las relaciones entre tablas permanezcan consistentes.

Esto evita:

- Datos huérfanos.
- Relaciones inválidas.
- Errores de integridad.

Beneficios generales de la seguridad implementada

La combinación de JWT, bcryptjs, middlewares y restricciones SQL permite:

- Protección de usuarios.
- Seguridad de credenciales.
- Control de accesos.
- Integridad de datos.
- Prevención de accesos no autorizados.
- Mayor estabilidad del sistema.

SQL utilizado para la creación de la Base de Datos

-- Crear base de datos

```
CREATE DATABASE IF NOT EXISTS tecdiagram;
USE tecdiagram;
```

TABLA: users

```
CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    firstName VARCHAR(255) NOT NULL,
    lastName VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    passwordHash VARCHAR(255) NOT NULL,
    role VARCHAR(50) NOT NULL DEFAULT 'student',
    createdAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

TABLA: classes

```
CREATE TABLE IF NOT EXISTS classes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    description TEXT,
    code VARCHAR(255) NOT NULL UNIQUE,
    ownerId INT NOT NULL,
    createdAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_classes_owner
        FOREIGN KEY (ownerId)
        REFERENCES users(id)
        ON DELETE CASCADE
) ENGINE=InnoDB;
```

TABLA: class_members

```
CREATE TABLE IF NOT EXISTS class_members (
    id INT AUTO_INCREMENT PRIMARY KEY,
    classId INT NOT NULL,
    userId INT NOT NULL,
    joinedAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY unique_member (classId, userId),
    CONSTRAINT fk_members_class
        FOREIGN KEY (classId)
        REFERENCES classes(id)
        ON DELETE CASCADE,
    CONSTRAINT fk_members_user
        FOREIGN KEY (userId)
        REFERENCES users(id)
        ON DELETE CASCADE
) ENGINE=InnoDB;
```

TABLA: designs

```
CREATE TABLE IF NOT EXISTS designs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    ownerId INT NOT NULL,
    classId INT NULL,
    content TEXT NOT NULL,
    image LONGTEXT,
    pdf_data LONGTEXT,
    description TEXT,
    isCopy BOOLEAN DEFAULT FALSE,
    originalId INT NULL,
    createdAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_designs_owner
        FOREIGN KEY (ownerId)
        REFERENCES users(id)
        ON DELETE CASCADE,

    CONSTRAINT fk_designs_class
        FOREIGN KEY (classId)
        REFERENCES classes(id)
        ON DELETE SET NULL,

    CONSTRAINT fk_designs_original
        FOREIGN KEY (originalId)
        REFERENCES designs(id)
        ON DELETE SET NULL
```

```
) ENGINE=InnoDB;
```

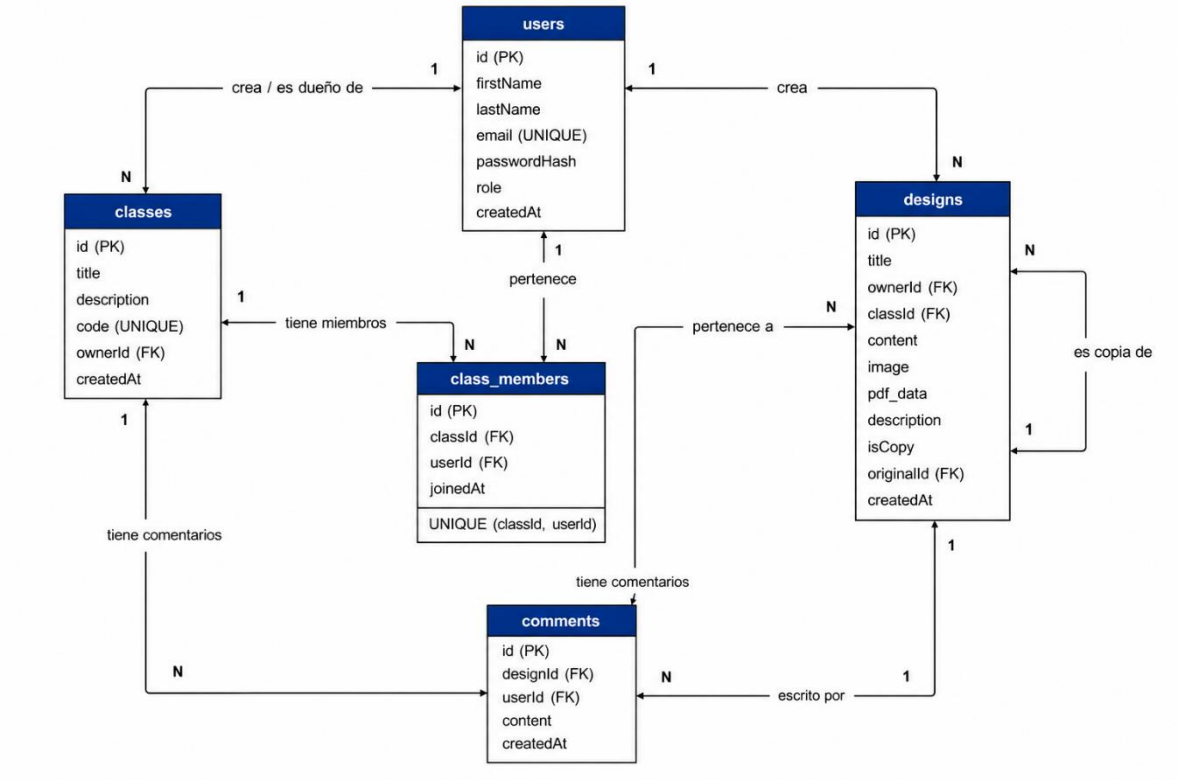
TABLA: comments

```
CREATE TABLE IF NOT EXISTS comments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    classId INT NOT NULL,
    userId INT NOT NULL,
    content TEXT NOT NULL,
    createdAt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_comments_class
        FOREIGN KEY (classId)
        REFERENCES classes(id)
        ON DELETE CASCADE,

    CONSTRAINT fk_comments_user
        FOREIGN KEY (userId)
        REFERENCES users(id)
        ON DELETE CASCADE
) ENGINE=InnoDB;
```

Diagrama DER (Entidad Relación



Relaciones:

USERS (1) ——— (N) CLASSES

USERS (1) ——— (N) CLASS_MEMBERS

CLASSES (1) ——— (N) CLASS_MEMBERS

USERS (1) ——— (N) DESIGNS

CLASSES (1) ——— (N) DESIGNS

DESIGNS (1) ——— (N) DESIGNS (originalId)

USERS (1) ——— (N) COMMENTS

DESIGNS (1) ——— (N) COMMENTS

USERS (N) ——— (M) CLASSES (vía CLASS_MEMBERS)

Diccionario de Datos

Tabla: **USERS**

Campo	Tipo	Clave	Descripción
id	INT	PK	Identificador único del usuario (auto incremental)
firstName	VARCHAR(255)		Nombre del usuario
lastName	VARCHAR(255)		Apellido del usuario
email	VARCHAR(255)	UQ	Correo electrónico único
passwordHash	VARCHAR(255)		Contraseña encriptada
role	VARCHAR(50)		Rol del usuario (student por defecto)
createdAt	TIMESTAMP		Fecha de creación

Tabla: CLASSES

Campo	Tipo	Clave	Descripción
id	INT	PK	Identificador único de la clase
title	VARCHAR (255)		Título de la clase
description	TEXT		Descripción de la clase
code	VARCHAR(255)	UQ	Código único de acceso
ownerId	INT	FK	Usuario creador (users.id)
createdAt	TIMESTAMP		Fecha de creación

Tabla: CLASS_MEMBERS

Campo	Tipo	Clave	Descripción
id	INT	PK	Identificador del registro
classId	INT	FK	Clase (classes.id)
userId	INT	FK	Usuario (users.id)
joinedAt	TIMESTAMP		Fecha de inscripción

Tabla: DESIGNS

Campo	Tipo	Clave	Descripción
id	INT	PK	Identificador del diseño
title	VARCHAR(255)		Título del diseño
ownerId	INT	FK	Usuario creador (users.id)
classId	INT	FK	Clase asociada (opcional)
content	TEXT		Contenido del diagrama
image	LONGTEXT		Imagen del diseño
pdf_data	LONGTEXT		PDF del diseño
description	TEXT		Descripción
isCopy	BOOLEAN		Indica si es copia
originalId	INT	FK	Diseño original (self FK)
createdAt	TIMESTAMP		Fecha de creación

Tabla: COMMENTS

Campo	Tipo	Clave	Descripción
id	INT	PK	Identificador del comentario
classId	INT	FK	Clase (classes.id)
userId	INT	FK	Usuario autor (users.id)
content	TEXT		Contenido del comentario
createdAt	TIMESTAMP		Fecha del comentario

Relaciones

Tabla Origen	Campo	Tabla Destino	Campo Referenciado	Tipo de relación	ON DELETE
classes	ownerId	users	id	1:N (usuario → clases)	CASCADE
class_members	userId	users	id	1:N (usuario → inscripciones)	CASCADE
class_members	classId	classes	id	1:N (clase → miembros)	CASCADE
Designs	ownerId	users	id	1:N (usuario → diseños)	CASCADE
Designs	classId	classes	id	1:N (clase → diseños)	SET NULL
Designs	originalId	designs	id	1:N recursiva (diseño → copias)	SET NULL
Comments	userId	users	id	1:N (usuario → comentarios)	CASCADE
Comments	classId	classes	id	1:N (clase → comentarios)	CASCADE

Derechos de Autor y Propiedad Intelectual

El sistema **DiagramTec** es un proyecto desarrollado íntegramente por el equipo de trabajo conformado por **Maite Vázquez, Joaquín Villalva, Santiago Álvarez, Leandro Pereyra y Nayla Gómez**. Tanto el nombre comercial ***DiagramTec***, su identidad visual, logotipo, diseño de la interfaz, estructura del sistema, documentación y código fuente original constituyen propiedad intelectual de sus autores.

Todos los derechos sobre el software se encuentran reservados. Queda prohibida la reproducción, distribución, modificación, publicación, comercialización o utilización total o parcial del proyecto sin la autorización expresa de los integrantes del equipo desarrollador.

El proyecto fue concebido inicialmente con fines académicos; sin embargo, se prevé su implementación en un servidor web para ofrecer el servicio al público. Asimismo, se contempla la incorporación de anuncios publicitarios como método de financiación para el mantenimiento y la evolución de la plataforma.

Todo el contenido original desarrollado específicamente para DiagramTec, incluyendo el código fuente, la base de datos, la documentación técnica, el diseño gráfico, la identidad visual, el logotipo y los recursos multimedia propios, pertenece exclusivamente al equipo desarrollador.

La utilización del sistema por parte de los usuarios no implica la cesión de ningún derecho de propiedad intelectual sobre el software. Los usuarios únicamente disponen del derecho de uso de la plataforma conforme a las condiciones establecidas por sus desarrolladores.

En caso de futuras versiones comerciales, actualizaciones o ampliaciones del sistema, los derechos de explotación, mantenimiento y evolución del proyecto continuarán perteneciendo al equipo creador, salvo que exista un acuerdo diferente formalmente establecido.

El equipo de desarrollo se reserva el derecho de realizar modificaciones, mejoras, correcciones y actualizaciones del sistema en cualquier momento con el objetivo de optimizar su funcionamiento, seguridad y experiencia de uso.

Licencias de Recursos Externos

Durante el desarrollo de DiagramTec se utilizaron distintos recursos de terceros para complementar las funcionalidades y el diseño del sistema. Estos componentes son utilizados respetando las licencias establecidas por sus respectivos autores.

Entre los principales recursos utilizados se encuentran:

- **Iconify**, utilizando colecciones de iconos **Tabler Icons**, **Fluent UI System Icons** y **Material Design Icons (MDI)**.
- La tipografía **Inter**, proporcionada por **Google Fonts**.
- Librerías y dependencias de código abierto instaladas mediante **npm**, incluyendo **React**, **Express** y otras necesarias para el funcionamiento del sistema.
- Servicios de autenticación mediante las **APIs de Google**, utilizando el protocolo **OAuth** para el inicio de sesión.

Todos estos recursos continúan siendo propiedad de sus respectivos autores u organizaciones y son utilizados conforme a las condiciones establecidas en sus licencias.

Los logotipos, imágenes principales, elementos gráficos personalizados y demás recursos originales de DiagramTec fueron creados por el equipo de desarrollo y forman parte de la propiedad intelectual exclusiva del proyecto.

Créditos del Proyecto

Proyecto: *DiagramTec*

Equipo de desarrollo:

- Joaquín Villalva
- Maite Vázquez
- Santiago Álvarez
- Leandro Pereyra
- Nayla Gómez

Estado del proyecto: En desarrollo, con proyección de implementación en un servidor web y futura disponibilidad para usuarios finales.

Copyright © 2026 – DiagramTec. Todos los derechos reservados.